

BLKR_v:基于 RISC-V 指令集架构的硬件仿真框架

摘要: 虚拟机以其低成本以及接近底层的优势被广泛应用于计算机教学领域,除此之外,虚拟机在环境隔离与环境迁移等场景也发挥了非常大的作用。多种架构,包括 x86, riscv, 均提供了虚拟化的加速能力,在保证虚拟机性能的同时,提供完整的计算机功能。针对当前教学领域中虚拟机抽象层次高,不利于初学者理解的问题,本文提出一种基于 verilog 的虚拟化方式,以使底层更接近真实硬件的表现形式,同时提供相应的软件环境支持模块验证。

本研究针对当前计算机教学中虚拟机抽象层级过高、硬件原理可视性不足的问题,提出基于硬件描述语言的虚拟化实现方案。基于 RISC-V 指令集架构并通过 Verilog 硬件描述语言,构建具备完整硬件行为可视化的教学型虚拟机系统,在保持现代计算机核心特征的前提下降低学习门槛,为计算机组成原理和操作系统等课程提供直观的实践平台。本研究的创新性工作主要体现在以下三个方面:

1、RISC-V 微架构设计与实现: 基于三级流水线结构设计 RV32I 兼容处理器核,采用 Verilog 实现具有完整指令执行能力的计算核心。通过集成 verilator 仿真环境构建指令级调试平台,并引入多线程协同验证机制提升仿真效率,确保每个时钟周期的硬件状态可观测。

2、混合式外设虚拟化方案: 创新性地采用宿主-目标协同架构,通过系统调用劫持和内存映射技术将输入输出设备抽象为虚拟外设模块。该设计在保持精简硬件规模的同时,完整支持存储设备虚拟化(Block Device)、终端交互(UART)等基础外设功能,实现了硬件资源占用量与功能完整性的平衡。

3、教学导向型操作系统开发: 研制面向教学验证的微内核系统,包含具有预取优化的二级页表内存管理、基于时间片轮转的多任务调度器,以及精简的文件系统。特别设计系统调用可视化追踪模块,可实时呈现从应用程序到硬件执行的完整调用链。

关键词: 硬件仿真; Risc-V; Verilog; CPU; 操作系统;

目 录

第 1 章 绪 论	1
1.1 研究背景及研究意义	1
1.2 国内外研究现状	2
1.2.1 国内研究现状	2
1.2.2 国外研究现状	2
1.3 研究目标和内容	3
1.4 论文结构安排	错误!未定义书签。
第 2 章 相关环境与工具	5
2.1 相关环境	5
2.2 相关工具	5
2.2.1 Risc-V 交叉编译链	5
2.2.2 GCC	6
2.2.3 Verilator	6
第 3 章 体系结构	8
3.1 层级间协议	8
3.1.1 指令集	8
3.1.2 外设驱动	12
3.1.3 系统调用	14
3.2 具体工程结构	17
3.3 系统整体架构	17
第 4 章 详细硬件设计	19
4.1 片上系统	19
4.1.1 取指单元	19
4.1.2 译码单元	20
4.1.3 执行单元	21
4.1.4 流水控制	22
4.1.5 CSRS 与 REGS	22

4.1.6 中断处理	23
4.2 外设	24
4.2.1 外设虚拟化	24
4.2.2 存储器模型	28
4.2.3 具体设备号及内存分布	28
第 5 章 操作系统详细设计	30
5.1 子系统设计	30
5.1.1 系统调用	30
5.1.2 文件系统	33
5.1.3 内存管理系统	36
5.1.4 I/O 子系统	41
5.1.5 进程管理	41
5.2 效果图演示	43
第 6 章 模块等价替换	46
6.1 文件系统等价替换实例	46
6.2 CPU 核替换	48
第 7 章 可视化工具设计	50
7.1 monitor 设备	50
7.2 数据可视化分析	50
参考文献	54

第1章 绪 论

1.1 研究背景及研究意义

随着计算机领域的技术迭代,体系结构的抽象层次逐渐增高[1],计算机的底层不再对用户开放,计算机初学者越来越难以理解硬件与软件的关系,因此各高校逐渐引入模拟器[2][3]进行教学,既能够起到教学作用,也能够节约成本,初学者也能理解抽象层次较低的模拟器。

然而,目前开源的模拟器中,gem5 着重于精确度和细节,提供了详细的体系结构模型和多种 CPU 模型,但其抽象层次仍然在硬件的上一层,且由于需要进行精细化的数据记录,性能受到影响而降低,且面向的用户是对体系结构较为熟悉的专家等;QEMU 是一个更通用的开源机器模拟器和虚拟化器[9][10][11],支持多种 CPU 架构并使用动态二进制翻译来执行模拟的指令,尽管性能得到了保证,但该模拟器仅仅是指令级模拟,并没有实际的流水线执行等工作。

现有的开源模拟器,如 gem5 和 QEMU,虽然在一定程度上提供了对计算机体系结构的模拟,但仍存在一定的局限性。Gem5 注重精确度和细节,提供了复杂的体系结构模型和多种 CPU 模型,适用于对体系结构有深入了解的专业人士。但由于其对数据记录的精细化要求,性能有所下降,并且其抽象层次仍停留在硬件之上,难以帮助初学者深入理解硬件的内部工作原理。QEMU 作为一种更通用的开源机器模拟器和虚拟化器,支持多种 CPU 架构,并通过动态二进制翻译执行指令,虽然能保证较好的性能,但仅进行指令级模拟,缺乏流水线执行等底层硬件行为的模拟,依然无法帮助初学者理解硬件与软件的深层联系。

RISC-V [4][5]作为完全开源的指令集架构(ISA),允许任何人自由使用而无需支付许可费用,这降低了硬件设计和软件开发的成本。其模块化设计使得 RISC-V 可以根据需求灵活扩展或缩减功能,适应从小型嵌入式系统到高性能服务器等各种应用场景[6][7][8]。因此,设计一个基于 RISC-V 的硬件仿真平台,能够为用户提供一个便捷的途径,帮助他们轻松理解计算机的整体架构。通过这样的平台,用户可以清晰地看到每一条指令在每个时钟周期(tick)中的执行过程和寄存器值变化。此外,基于计算机体系结构,平台还支持各子系统的替换和定制,帮助用户验证自己的设计思想和算法。

综上所述,研究和设计一个基于 RISC-V 指令集的硬件仿真平台,不仅能够帮助初学者快速理解计算机硬件的工作原理,还能促进计算机教学方式的创新和变革。它不仅

突破了传统教学中仅依赖书本的局限，更为学生提供了一个灵活的实验环境，以深入探索计算机体系结构的本质。

1.2 国内外研究现状

1.2.1 国内研究现状

国内关于仿真平台的研究与开发主要集中于对某个特定的知识领域进行建模，例如电子设计中对 EDA 教学平台的建设，提出基于 EDA 的一体化、网络化解决方案，结合虚拟实验室理论和仿真技术，以图形、数字、曲线等形式即时显示理论课程中难以用语言、文字表达的复杂现象的变化过程集成计算机硬件教学中各门课程、各种技术并统筹考虑，构成内容丰富、实验、仿真相结合的教学仿真平台，给出仿真结果[12]。或者针对理论教学而提出的概念，在计算机虚拟仿真实验平台上教师能够将传统实验题目转变为自主型实验题目并结合实验内容、实验要求、机房环境、网络虚拟平台建设情况来设置实验题目将传统实验题目改为以系列问题解决为主线的任务型情境引导学生结合自身学习情况来选择实验实现对实验的自主设计培养学生的实验探究能力[13]。

而关于通用处理器模拟器的研究，NEMU 是一款简单但完整的全系统仿真器，专为教学目的而设计。其性能接近 qemu，底层使用单指令执行来模拟 cpu 的运行。带有简单调试器的小型显示器，支持 x86、mips32、riscv32 和 riscv64 等指令，拥有 TLB，外设以及 I/O 等基础设施。但其技术实现存在显著局限性：首先，仿真器底层采用高级语言（C/C++）构建硬件行为模型，导致学生无法直接观察微架构级信号交互；其次，外设模拟依赖虚拟化技术抽象实现，与真实硬件驱动开发流程存在差异，易造成学习迁移障碍。此外，国内高校近年尝试将 FPGA 引入计算机组成课程实验，例如基于 Vivado 搭建的 RISC-V 软核实验框架，但其设计重点在于硬件逻辑验证，缺乏对操作系统、应用程序等上层软件栈的支持，难以形成完整的“从晶体管到应用程序”教学链路。

1.2.2 国外研究现状

在硬件仿真框架方面，有研究者设计了行为级综合框架 ASSIST，可以根据对指令集的功能描述，协助生成一系列 RISC-V 处理器，满足不同微架构设计选择的需要。还有研究者提出了 FPGA 软核处理器的可重构架构，用于实现基于 SRAM 的 FPGA 软核处理器。

GVSoc[14]是一个高度可配置、快速且准确的事件驱动模拟器，它结合了 C++模型

的效率和 Python 配置脚本的灵活性。GVSoC 完全开源，旨在推动 RISC-V 基于 IoT 处理器的并行和异构研究，它具有三个基础特征：基于 Python 的硬件描述模块配置、平台参数的易校准以实现准确的性能估计、高速仿真。实验结果表明，GVSoC 能够在全平台级别（处理器、内存、外设和 IO）进行实际的功能和性能分析以及设计探索，与周期精确仿真相比，速度提升了 2500 倍，性能分析的误差通常低于 10%。然而，GVSoC 的抽象层级仍停留在总线事务级别，无法呈现处理器内部流水线冲突、缓存一致性协议等关键微架构行为，且配置脚本的复杂性对教学场景下的快速部署构成挑战。

Unlocking the Potential of RISC-V Heterogeneous MPSoC: A PANACA-Based Approach to Simulation and Modeling[15]: 该研究扩展并结合了 PANACA 仿真平台与 RISC-V 虚拟原型，创建了一个高效的、灵活的 MPSoC 仿真平台，该平台使用 SystemC-TLM 编写。这个仿真平台能够运行各种应用程序，例如机器学习应用，并能支持如 FreeRTOS 这样的实时操作系统。开发的内存映射网络适配器提供了 RISC-V 处理器和 NoC 之间的无缝接口，并通过统一 API 实现组件之间的高效通信。这个 API 的设计使得使用它的应用程序可以应用于仿真和硬件，而无需更改应用程序代码。该仿真平台经过广泛的测试，证明了其作为早期硬件开发和研究的宝贵工具的准确性和可靠性。但其设计目标偏向于嵌入式系统开发，默认外设驱动接口与通用操作系统（如 Linux）存在显著差异，且缺乏对虚拟内存管理、系统调用中断等核心机制的可视化支持，难以满足计算机系统原理教学中对“全栈透明性”的需求。

综合国内外研究成果，当前教学仿真平台存在以下核心缺陷：

1、抽象层级失衡：高级语言实现的指令集模拟器（如 QEMU）屏蔽硬件细节，而 RTL 级模型（如 FPGA 工程）又过度聚焦电路实现，均无法同时满足“硬件行为可见性”与“软件生态完整性”的教学需求。

2、功能碎片化：现有工具多专注于处理器核仿真或操作系统移植的单一环节，缺乏从 CPU 微架构、外设驱动到系统调用链路的端到端可视化支持。

3、学习曲线陡峭：GVSoC 等工具依赖脚本配置与领域特定语言（DSL），NEMU 等开源项目代码规模庞大（超 10 万行），导致学生陷入工程复杂性而非原理性学习。

1.3 研究目标和内容

针对上述问题，本研究提出构建“全栈透明”的 RISC-V 教学虚拟机系统（简称 EduVM），其核心目标在于：通过硬件描述语言（Verilog）实现从处理器微架构到操作

系统内核的逐层可视化建模，在保留现代计算机系统核心特征的前提下，以模块化设计降低学习复杂度，形成“硬件行为可观测—软件机制可追溯”的闭环教学链路。具体研究内容包括：

1、可观测型 RISC-V 处理器核设计

基于三级流水线实现 RV32I 指令集硬件原型，通过非侵入式探针模块捕获流水线级间寄存器、ALU 操作数等关键信号，生成带时序标注的执行轨迹。集成 Verilator 仿真框架实现周期精确仿真。

2、轻量化混合虚拟化外设架构

提出“硬件建模+宿主代理”的混合外设方案：中断控制器、UART 等基础外设通过 Verilog 实现精确时序模型，而磁盘、网络等复杂设备则映射至宿主机文件系统与 Socket 接口。

3、教学适配型操作系统开发

实现微内核，集成物理内存分页、进程调度与文件系统模块，关键系统调用路径植入调试桩函数，可动态输出从用户态 API 调用到硬件中断响应的全链路日志。开发可视化监控工具链，支持实时展示页表映射关系、进程状态机迁移、系统调用栈回溯等核心机制，并与处理器仿真波形实现时序同步关联。

第 2 章 相关环境与工具

2.1 相关环境

该系统运行的硬件环境如表 2-1

表 2-1 硬件环境

硬件名称	型号
基座	虚拟机
处理器	2 处理器，4 核
内存	4GB
磁盘	100GB

该系统的开发工具、语言及其版本如表 2-2

表 2-2 软件环境

软件名称	版本
操作系统	Ubuntu 22.04 无图形化版本
GCC	11.4
Verilator	5.035
Riscv32-unknown-elf-gcc	14.2.0, rv32i, ilp32
fltk	
GNU make	4.3

2.2 相关工具

2.2.1 Risc-V 交叉编译链

交叉编译链是一套用于在一种计算机平台（称为“宿主平台”，如 x86 架构的 PC）上生成另一种不同架构平台（称为“目标平台”，如 RISC-V 架构的嵌入式设备）可执行代码的软件开发工具集合。与本地编译（在相同平台编译和运行）不同，交叉编译链使开发者能够在高效或资源丰富的环境中开发和调试程序，最终生成适用于资源受限或异构硬件平台的二进制文件。交叉编译链通常包括交叉编译器、交叉链接器、交叉汇编器，目标平台库与头文件。

此处使用的 riscv 交叉编译链主要用于，将操作系统编译为自定义的架构平台的 ELF 二进制文件，即支持 rv32i 整数指令集，mabi 为 ilp32（不含浮点）的目标可执行文件，

从而在 BLKRV 上进行执行。

除了仅仅编译交叉编译链以外，还对该编译链做了部分修正，包括将浮点库进行置换，以同时支持 IEEE 754 浮点格式以及 posit 浮点格式。

2.2.2 GCC

GCC 是由 GNU 项目开发的开源编译器集合，最初仅支持 C 语言（GNU C Compiler），后扩展为支持 C++、Fortran、Ada、Go 等多种编程语言的跨平台编译工具。其核心架构分为前端（解析源代码生成中间表示）、中端（语言无关的优化）和后端（生成目标平台机器码），模块化设计使其能灵活适配不同硬件架构（如 x86、ARM、RISC-V）。GCC 是交叉编译链的核心工具，通过配置目标平台的三元组（如 arm-linux-gnueabi），可在高性能宿主主机上为嵌入式设备等异构硬件生成高效代码，同时提供多级优化选项（如 -O3 激进优化、-Os 精简代码）。尽管在编译速度和错误提示上逊于 LLVM/Clang 等新兴工具，GCC 仍凭借其稳定性、广泛的平台支持（如 Linux 内核开发）及深度优化能力，在嵌入式开发、操作系统构建和高性能计算领域占据重要地位，持续演进支持新兴架构（如 RISC-V）和静态分析等现代特性。

在 BLKRV 系统中，主要使用 g++ 将外设部分代码编译成可链接的静态文件，与 verilator 编译得到的静态文件进行链接得到最终的可执行文件，也即虚拟机本体。

2.2.3 Verilator

Verilator 可将硬件描述语言 Verilog/SystemVerilog 编译成优化后的支持多线程的高级语言 C++/SystemC 模型，通过在高级语言中例化硬件描述语言，进而使用高级语言的特性实现对 Verilog/SystemVerilog 的验证。简单来说，Verilator 对 Verilog/SystemVerilog 等硬件描述语言添加了一个基于 C++/SystemC 的 wrapper，而后借助 C++ 和 Verilator 的特定库实现对 Verilog/SystemVerilog 的高效仿真。Verilator 本质上是一个 Verilog/SystemVerilog 模拟器。它具有商业级、超快速、免费和开源等特性，但是它不能直接替代 Modelsim、Questa Sim、Synopsys VCS、Vivado Xsim 和其他基于事件的模拟器。Verilator 是一个基于周期的模拟器，这意味着它不能在单个时钟周期内计算时间，也不能模拟精确的电路时序。因此，在 Verilator 中电路状态通常每个时钟周期评估一次，因此不能观察到任何周期内的小故障，并且不支持定时信号延迟。因此 BLKRV 系统仅仅处于“逻辑正确”，而不是“物理正确”，也如同研究目标所说，BLKRV 仅仅是提供一个学习的平台。注意到 Verilator 编译出来的库是需要一个 Wrapper 的，简单讲，即是

信号输入，BLKRV 将 Wrapper 与外设代码进行结合从而实现完整的模拟。

第 3 章 体系结构

3.1 层级间协议

3.1.1 指令集

指令集作为底层硬件与上层软件之间的协议，在整个设计中占据了相当重要的位置，因此此处将详细介绍所有的相关指令以及其相关的功能。

BLKRV 采用的是基础的整数指令集，以及必要的 csr 指令，riscv 的指令分为六个类型，如表 3-1。

表 3-1 指令集分类

目的	类型
用于寄存器-寄存器操作	R 类型指令
用于短立即数和访存 load 操作	I 型指令
用于访存 store 操作	S 型指令
用于条件跳转操作	B 类型指令
用于长立即数	U 型指令
用于无条件跳转	J 型指令

对于所有的基础整数指令的解析如表 3-2。

表 3-2 rv32i 基础指令集

汇编格式	功能
lui rd, imm	加载高位立即数（20 位）
auipc rd,imm	PC+偏移量（构造地址）
jal rd,offset	PC 相对跳转（保存返回地址）
jalr rd,offset(rs1)	寄存器间接跳转
beq rs1,rs2,offset	相等时跳转
bne rs1,rs2,offset	不相等时跳转
blt rs1,rs2,offset	小于时分支（有符号）
bge rs1,rs2,offset	大于等于时分支（有符号）
bltu rs1,rs2,offset	小于时分支（无符号）
bgeu rs1,rs2,offset	大于等于时分支（无符号）
lb rd,offset(rs1)	加载字节（符号扩展）

汇编格式	功能
lh rd,offset(rs1)	加载半字（符号扩展）
lw rd,offset(rs1)	加载字（32 位）
lbu rd,offset(rs1)	加载字节（无符号）
lhu rd,offset(rs1)	加载半字（无符号）
sb rs2,offset(rs1)	存储字节
sh rs2,offset(rs1)	存储半字
sw rs2,offset(rs1)	存储字
addi rd,rs1,imm	加立即数
slti rd,rs1,imm	立即数置位小于（有符号）
sltiu rd,rs1,imm	立即数置位小于（无符号）
xori rd,rs1,imm	立即数异或
ori rd,rs1,imm	立即数或
andi rd,rs1,imm	立即数与
slli rd,rs1,imm	立即数逻辑左移
srlr rd,rs1,imm	立即数逻辑右移
srai rd,rs1,imm	立即数算术右移
add rd,rs1,rs2	寄存器相加
sub rd,rs1,rs2	寄存器相减
sll rd,rs1,rs2	逻辑左移
slt rd,rs1,rs2	置位小于（有符号）
sltu rd,rs1,rs2	置位小于（无符号）
xor rd,rs1,rs2	按位异或
srl rd,rs1,rs2	逻辑右移
sra rd,rs1,rs2	算术右移
or rd,rs1,rs2	按位或
and rd,rs1,rs2	按位与
ecall	环境调用（系统请求）
nop	空操作（伪指令）

对于所有的 csr 指令的解析如表 3-3。

表 3-3 rv32csr 指令集

汇编格式	功能
csrrw rd,csr,rs1	原子读后写 CSR
csrrs rd,csr,rs1	原子读后置位 CSR 位
csrrc rd,csr,rs1	原子读后清除 CSR 位
csrrwi rd,csr,rs1	立即数版本 CSRRW（5 位零扩展）
csrrsi rd,csr,rs1	立即数版本 CSRRS
csrrci rd,csr,rs1	立即数版本 CSRRC

对于所有指令的二进制编码格式如表 3-4。RISC-V 作为开源指令集架构（ISA），凭借其独特的设计理念和技术特性，成为计算机系统教育的理想选择。RISC-V 采用 BSD 开源协议，允许任何组织或个人自由使用、修改和分发，无需支付授权费用。规避商业 ISA（如 ARM/x86）的授权限制，教育机构可自由构建全栈教学平台（从 CPU 设计到操作系统）。学生能够完整接触工业级 ISA 规范，避免传统教学中“黑盒化”的指令集模拟器限制。

本项目的 RV32I 处理器核基于官方手册实现，学生可直接对照标准文档调试流水线冲突。学生可先实现 RV32I 核心，逐步添加 M 扩展实现乘法器、A 扩展支持多线程同步。支持添加自定义扩展（如本项目的 Posit 浮点指令），培养硬件-软件协同设计能力。在本项目中，学生通过禁用 M 扩展，手动实现移位加法乘法电路，深入理解算法优化。在 RISC-V 中，单周期处理器仅需 3 级流水线即可实现完整功能（对比 x86 需 15+级），降低 FPGA 实验门槛。规整的指令格式使反汇编代码更易解读（如表 3-4 所示），学生可手动跟踪指令执行流程。

RISC-V 通过其开源基因、模块化设计及精简架构，完美契合计算机系统教育对透明度、可扩展性和实践性的核心诉求。在本项目中，学生从 Verilog 编码 RV32I 核开始，逐步构建系统，最终在 os 上运行应用程序，完整经历“门电路→指令集→操作系统→应用生态”的全栈开发流程。这种深度参与的学习模式，使抽象概念具象化，复杂系统层次化，有效缩短了理论认知与实践技能之间的鸿沟。

表 3-4 指令二进制编码格式

		操作码	指令
imm[31:12]	rd	0110111	LUI
imm[31:12]	rd	0010111	AUIPC

					操作码	指令
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0100000	shamt	rs1	101	rd	0010011	SRLI
0000000	shamt	rs1	101	rd	0010011	SRAI
0100000	rs2	rs1	000	rd	0110011	ADD
0000000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0000000	rs2	rs1	101	rd	0110011	SRA
0100000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND
0000000	00000	00000	000	00000	0001111	ECALL
csr		rs2	001	rd	1110011	CSRRW
csr		rs2	010	rd	1110011	CSRRS
csr		rs2	011	rd	1110011	CSRRC
csr		rs2	101	rd	1110011	CSRRW1

				操作码	指令
csr	rs2	110	rd	1110011	CSRRS1
csr	rs2	111	rd	1110011	CSRRC1

3.1.2 外设驱动

在 BLKRv 中设计了八种较为常见的设备，包括 I/O 设备，存储器设备等，为了简化开发以及方便初学者理解，BLKRv 将外设的调用进行了尽可能的简化，抛开了现代的所有的相關外设的公有协议，为每个设备设计了其对应的控制寄存器以及缓存，并将驱动代码直接写入内核，采用这种原始的方式，能够将工程的结构尽可能简单化。

1. bios 固件

BIOS（基本输入/输出系统）是计算机启动时运行的固件程序，通常存储在主板芯片中。其主要负责执行硬件自检（POST）、初始化 CPU、内存、硬盘等关键组件，并引导操作系统加载。作为硬件与操作系统之间的桥梁，BIOS 提供基础硬件控制接口，例如通过中断服务程序管理输入输出操作。传统 BIOS 受限于 16 位架构和 MBR 分区，而现代 UEFI（统一可扩展固件接口）支持 64 位、安全启动、GPT 大容量硬盘等特性，成为当前主流的启动标准。BLKRv 中该设备属于纯粹的存储器设备，驱动仅需要响应读取地址即可。pc 寄存器初始为 0，程序将从 bios 的 0 地址开始执行。

2. 内存

RAM（随机存取存储器）是计算机中用于临时存储运行中程序和数据的高速易失性存储器，允许 CPU 快速读写信息。其特点是存取速度远超硬盘（纳秒级响应），支持直接访问任意存储单元，但断电后数据立即丢失。作为系统性能关键指标，RAM 容量决定同时处理任务的能力，现代计算机通常配备 8-64GB 的 DDR4/DDR5 内存模块，通过多通道技术提升带宽，配合 CPU 高速缓存实现高效数据处理。BLKRv 中该设备属于纯粹的存储器设备，驱动仅需要响应读取地址即可。可扩展内存控制器，需要注意某些耦合设备实现，例如 ram 等。

3. 硬盘

硬盘是计算机中用于长期存储数据的非易失性存储设备。HDD 通过磁头在高速旋转的磁盘盘片上读写数据，提供大容量（最高达 20TB）和低成本特性，但速度较慢（约 100-200MB/s）且抗震性差；SSD 采用闪存芯片存储，无机械结构，读写速度可达 3500-7000MB/s（NVMe 协议），具有低延迟、高抗震和静音优势，但单位容量成本较高。现代硬盘通过 SATA 或 PCIe 接口与主板连接，支持 TB 级存储容量（常见 512GB-8TB），

结合 3D NAND 堆叠技术持续提升存储密度，是操作系统、应用程序和用户文件的永久存储载体。BLKRV 中该设备属于纯粹的存储器设备，驱动仅需要响应读取地址即可。键盘按下时产生中断，0x10100000 地址处将会存储三个字符缓存。

4. 键盘

没有为键盘设计控制寄存器，当存在输入时，将内容写入到映射的内存空间地址，此时触发中断信号，系统执行相应的系统调用，将键盘的缓存写入到系统的缓存当中。

5. 字符输出设备（screen）

该设备具备一个控制寄存器，该寄存器用于控制字符设备是否显示内容，当系统将缓存写入设备的缓存区时，向控制寄存器写入内容，将会触发设备的显示功能。0x10200003 写入 1 时触发输出，screen 设备将会将 0x10200004 后的字符输出到宿主机终端。

6. 网卡

NIC 的驱动协议如表 3-5。该设备可以理解为对 socket 的简化封装，也可以自己实现完整的计算机网络体系结构内容得到更高的网络带宽。nic 设备采用 dma，直接操作 ram。

表 3-5 NIC 功能驱动协议

写入信号	对应功能	数据寄存器功能与地址偏移
1	Accept	0-16:ip;20:port;24:protocol;28:sockfd
2	Send	4:sockfd;8:send_data_ram_addr;12:data_len
3	Recv	4:sockfd;8:send_data_ram_addr;12:data_len
4	Connect	0-16:ip;20:port;24:protocol;28:sockfd
5	Close	4:sockfd

7. GPU

此处的 GPU 实际上仅仅是一个画图接口设备，BLKRV 并没有实现完全的渲染流程，仅仅是帮助系统具备图形化的能力，提供了双缓冲机制，存在一个控制寄存器，两个数据缓冲区。gpu 设备并不具备通用并行计算能力，但是可以扩展，绘图功能被限制，也可以扩展，该设备贴图时为了性能需求，将会直接从 flash 读取文件内容，这与文件系统强绑定。对应信号和功能如表 3-6。

表 3-6 GPU 功能驱动协议

写入信号	对应功能	数据寄存器功能与地址偏移
1	绘制文字	目标 ip
2	绘制三角形	目标端口
3	刷新屏幕	发送数据长度
4	绘制 jpg	发送数据缓存区
5	绘制 png	接收数据长度

8. Monitor

此处 Monitor 设备可以理解为一个硬件数据采集卡，运行时 SOC 将会提供一个接口可以读出对应的硬件数据，此时 monitor 收集这些数据并存储到宿主机。0x60300000 为控制寄存器地址，当向控制寄存器地址写入 1 时触发监测器的启动，此时开始进行性能数据的采集。写入 1 后写入 0，将会触发监测器的关闭，并将数据存储至宿主机。

3.1.3 系统调用

系统调用（System Call）是操作系统内核为应用程序提供的标准化服务接口，允许用户程序安全地访问硬件资源和核心功能（如文件操作、进程管理、网络通信）。当应用程序需要执行特权操作（如读写文件、创建进程）时，会通过软中断（如 x86 的 `int 0x80` 或 RISC-V 的 `ecall` 指令）从用户态切换到内核态，由操作系统内核验证权限后执行具体操作（如磁盘驱动调用、内存分配），最后将结果返回用户空间。常见的系统调用包括 `open()` 打开文件、`fork()` 创建进程、`send()` 发送网络数据等，是隔离用户程序与底层硬件、保障系统安全稳定的核心机制。系统调用是用户程序与操作系统的桥梁，具体来讲，即操作系统提供的访问硬件的能力，所有的操作系统不应让用户直接操作硬件，一方面是安全性问题，另一方面在于减小用户的开发压力，避免让用户面对设备驱动等具体的细节，因此，系统调用几乎是操作系统的核心功能，此处设计了尽可能多的系统调用以覆盖用户的常规需求。

1. exit

函数原型：int exit();

功能：退出进程，该系统调用一般由编译器自动生成在 `_start` 段中，但是由于 BLKRV 系统并不是常规的 linux，因此实现了一个 `_start` 段，用户编写程序时需要携带该段，以调用 `exit` 以及进入 `main` 函数入口等功能。`exit` 需要清理用户程序的内存空间回收所有的相关资源。

2. exec

函数原型: `int exec(uint32_t inode_id,int stdout,char** para,uint32_t para_num);`

功能: 执行程序, 需要传入文件的 `inode_id`, 该 `id` 由 `open` 调用获得, `stdout` 指定了该进程的标准输出是位于文件或者直接输出到屏幕; `para` 是传入参数的指针, `para_num` 是传入参数的数量。有趣的是, 该函数可以在用户程序中执行程序本身, 出现程序递归的现象。

3. vgetch

函数原型: `char vgetch();`

功能: 获取一个输入字符。

4. kbhit

函数原型: `int kbhit();`

功能: 检测是否有键盘按下。

5. poweroff

函数原型: `int powoff();`

功能: 发送关机信号。

6. vprint

函数原型: `int vprint(char* str,uint32_t length);`

功能: 打印内容, 输入需要打印字符串的地址, 以及对应的字符串长度。

7. read

函数原型: `int read(uint32_t inode_id,char* buf,uint32_t start,uint32_t count);`

功能: 从文件中读取内容, 存储到 `buf` 缓存中, 可以指定 `start` 开始字节, 该参数直接替代文件指针, 可以更加灵活的操作文件。需要指定读取的大小。

8. write

函数原型: `int write(uint32_t inode_id,char* buf,uint32_t start,uint32_t count);`

功能: 将缓存 `buf` 写入文件, 参数同 `read`。

9. create

函数原型: `int create(char* file_path,char type,uint32_t* inode_id);`

功能: 根据传入的 `file_path` 文件路径创建一个文件, 需要指定文件类型 (文件夹/文件), 需要传入 `inode_id` 指针以获取 `id`, 因为该函数还需要返回创建文件状态, 包括是否创建成功, 或者是否重名等等。

10. open

函数原型: `int open(const char* file_path);`

功能: 通过传入的文件路径打开某个文件, 返回文件 id。

11. free

函数原型: `int free(void* pointer);`

功能: 释放空间, 传入指针。

12. malloc

函数原型: `void* malloc(uint32_t size);`

功能: 分配空间, 传入大小, 返回首地址。

13. send

函数原型: `int send(socket* sock,char* buf,uint32_t buf_length);`

功能: 将 buf 发到目标设备, sock 包含 ip 和 port, 需要指定发送的数据长度。

14. recv

函数原型: `int recv(socket* sock,char* buf,uint32_t buf_length);`

功能: 从目标设备接收数据。

15. draw_label

函数原型: `int draw_label(point* p,char* str,color* c,int font);`

功能: 在图形化界面写文字, 并没有处理字模等更复杂的东西, 这里仅仅提供一个绘制文字的接口。

16. draw_triangle

函数原型: `int draw_triangle(point* p1,point* p2,point* p3,color* col);`

功能: 在图形界面绘制三角形, 需要传入三个顶点的坐标, 同时传入三角形的颜色, color 由三原色组成。

17. flush

函数原型: `int flush();`

功能: 刷新界面。

18. open_monitor

函数原型: `int open_monitor();`

功能: 打开监测器, 可在函数前后调用以查看函数的具体性能数据。

19. close_monitor

函数原型：int close_monitor();

功能：关闭检测器，该操作将会在本地产生成数据文件。

3.2 具体工程结构

BLKRV 系统所有的部件都由代码编写而成，因此确定各部件间的协作方式尤为重要，这将决定了系统的运行方式以及性能几何。

在系统中，出于开发难度和开发成本考虑，并不对所有硬件进行“物理化”，也就是并不是所有的硬件都使用 Verilog 语言进行编写，其中前面提到的外设自然由 c++ 进行编写，除此之外，片上系统的内存管理单元（MMU），以及总线设备（协议采用 RIB）均被虚拟化，也就是外设、MMU、RIB-BUS 采用 c++ 进行编写。

将编写完成的硬件代码由 Verilator 编译后，将会生成相应的库文件，也即可链接的

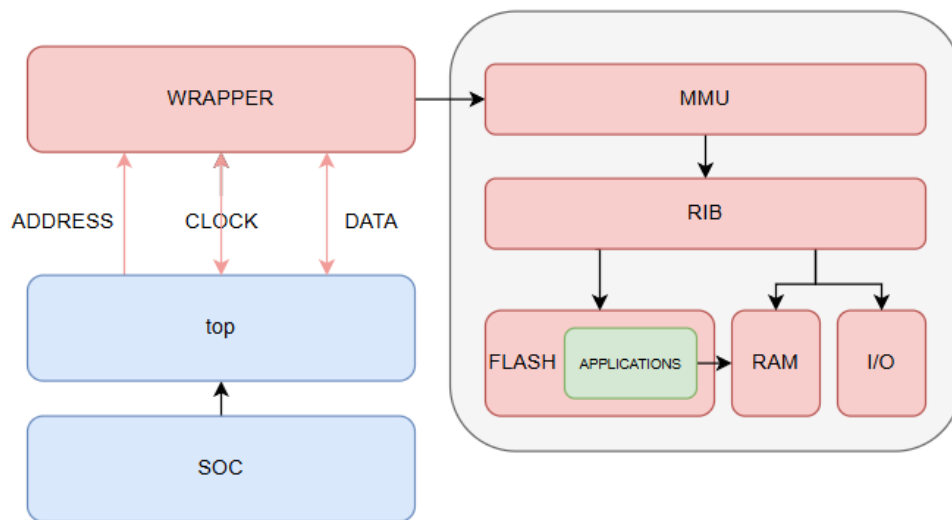


图 3-1 具体工程结构图

输出文件，该库文件向使用者提供了一个 Vtop 模块，通过该 Vtop 模块，可以访问到代码中的 top 部件，这是向外暴露的一个层级，可以理解为硬件上的各式接口，通过访问这些接口，可以得到此时硬件向外输出的数据，从而做出响应。注意到 top 暴露了一个 clk 的数据接口，通过改变这个数据接口的值，在高平信号和低平信号之间切换，即可实现运行 CPU 的效果。具体的工程结构如图 3-1。

3.3 系统整体架构

BLKRV 系统遵循传统的计算机体系结构，在细节上做了一些简化处理，其余大部

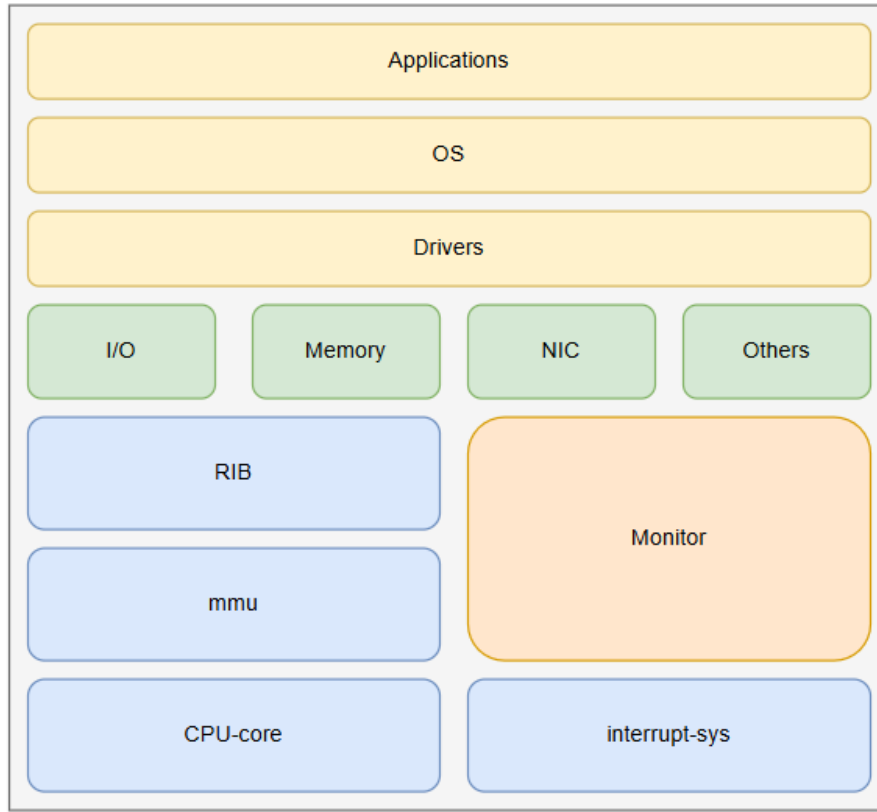


图 3-2 系统总体结构图

分基本遵照常规的处理流程，此外，BLKRV 增加了一个 monitor 设备，该设备在 BLKRV 中不属于常规的 device，而是分析获取上下硬件的数据。MMU 设备中并不包含三级缓存结构用于访存加速，一方面是因为开发难度，包括需要考虑缓存一致性，换入换出策略，以及配合操作系统完成复杂的数据获取操作，另一方面在于虚拟设备从“文件存储器”中获取数据的速度并不能比得上实际硬件，因此此处如果再增加一层抽象，将极大地影响系统的整体性能。系统的总体架构如图 3-2。

第 4 章 详细硬件设计

4.1 片上系统

对于整个系统，其数据流图如图 4-1。其中模块包含了控制单元，计算单元和存储单元。IFU（指令获取单元）→IDU（指令解码单元）→EXECU（执行单元）构成三级流水线：IFU 从内存读取指令，IDU 解析操作码与操作数，EXECU 协调 ALU（算术逻辑单元）、TRANS（数据搬运模块）执行分支操作。关键组件中，MMU（内存管理单元）通过 RIB（总线接口）连接外部设备，负责虚拟地址转换与 DMA 控制；REGS（通用寄存器组）与 CSRS（控制状态寄存器）双向交互执行单元，保存运算中间状态；interrupt-Ctrl（中断控制器）接收外设信号，触发流水线暂停与异常处理。黄色数据流贯穿指令处理全链路，红色信号流则驱动各模块协同（如时钟同步、中断响应），完整呈现了从指令获取到硬件交互的计算机系统运作逻辑。

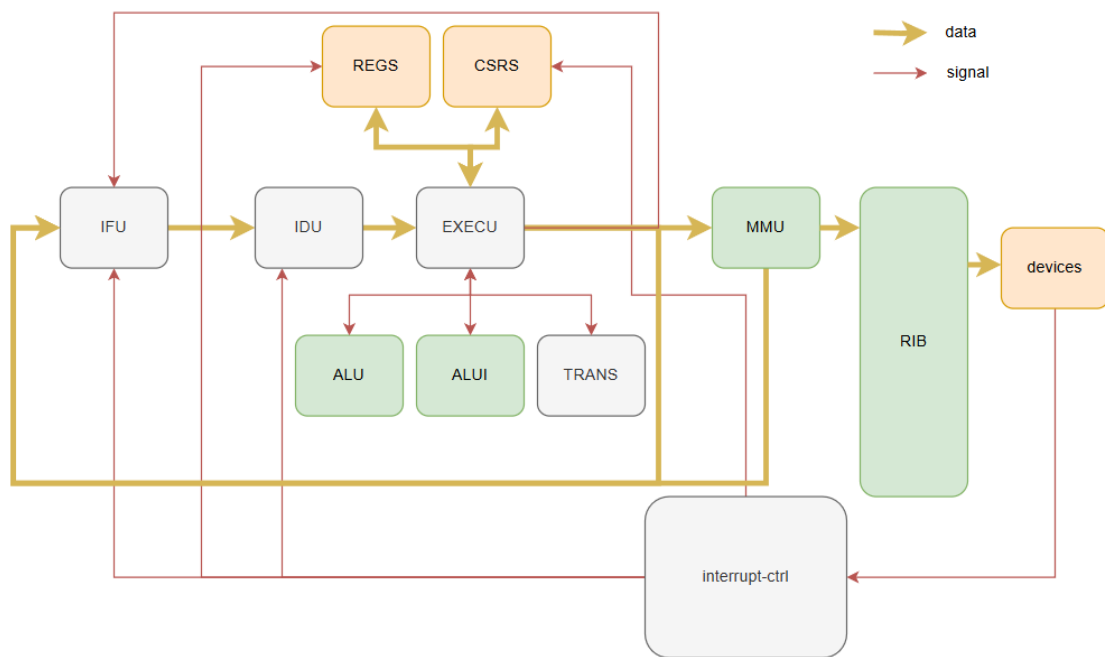


图 4-1 整体数据流图

4.1.1 取指单元

取值模块的输入输出如图 4-2，取值单元由于维护了 pc 寄存器，也即计数寄存器，因此它在各个流程中均会出现，但同时也为整个流水线带来一定的复杂性。

该模块在上升沿触发，信号的优先级依次是 $\text{interrupt_flag_i} > \text{syscall_flag_i} > \text{mret_flag_i} > \text{sret_flag_i} > \text{jump_flag_i} > \text{hold_flag_i}$ ，当 interrupt_flag_i 为高平信号时，

代表此时触发中断，pc 寄存器写入 mtvec，mtvec 寄存器存储的是中断调用查询程序的物理地址；当 syscall_flag_i 为高平信号时，代表触发系统调用，此时 pc 寄存器写入 stvec，stvec 存储的是系统调用查询程序的物理地址；mret_flag_i 代表中断程序执行返回，此时需要恢复到系统之前执行的地址，该值被存储到 mepc 寄存器中；sret_flag_i 同上；jump_flag_i 代表的是跳转信号，pc 寄存器写入 new_pc_i 的值，同时更新 pc_change_flag_o 为 2'b10，因为跳转需要清空后两个时钟周期，也即后两个时钟周期不能读取新的指令，否则会产生流水错误；hold_flag_i 信号代表流水线停止，此时 pc 寄存器需要减 4 以防止当前指令被冲刷。

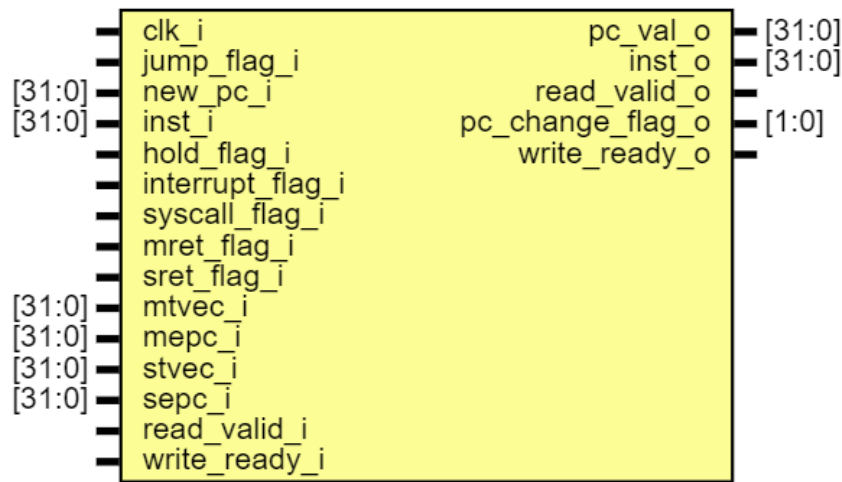


图 4-2 取指模块输入输出

4.1.2 译码单元

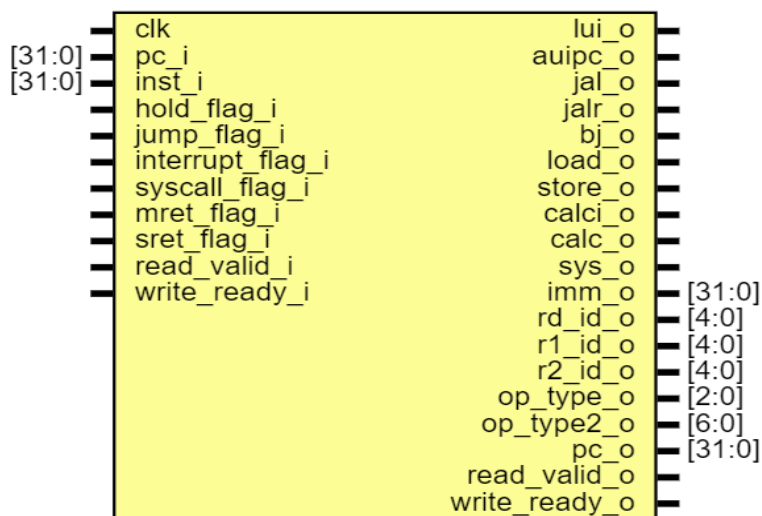


图 4-3 译码模块输入输出

图 4-3 为译码模块的输入输出，译码模块主要用于解析取指模块得到的指令，并产生相应的信号传递给下一级，仍然是上升沿触发，值得注意的是，对于除开指令输入以外的输入，该模块都将冲刷所有的现有信号，以支持流水。

译码模块的主要功能参照图 3-1 二进制指令格式进行译码工作。

4.1.3 执行单元

执行单元输入输出如图 4-4，其主要功能是将需要处理的数据传递到各功能模块，包括 alu 计算模块，alui 立即数计算模块，trans 跳转指令处理模块，以上模块均是组合逻辑电路，因此不参与流水线控制，同时执行单元还负责了通用寄存器与 CSR 寄存器的读出写入。实际的 Verilog 代码中，并没有事无巨细地实现包括全加器等等的电路部件，均是直接使用内部继承的组件，同时具有高性能与简洁两个优点。

同时还需要注意到的是，exu 单元还同时产生新的跳转地址，当面临 jal 指令时，产生的新 pc 值需要由 pc_o 与立即数相加得到，pc_o 值由 trans 单元产生，pc 值在流水线上传递，当来到，对于 jalr 指令，即是 r1 寄存器与立即数相加得到，对于条件跳转指令，直接赋值为 trans 产生的新 pc 地址。以上信号将会触发 jump_flag。

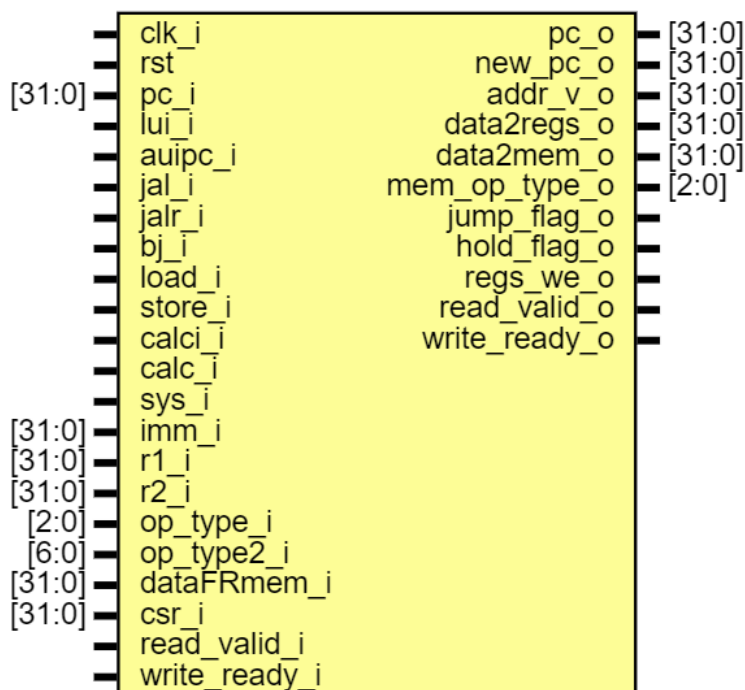


图 4-4 译码单元

4.1.4 流水控制

流水控制单元输入输出如图 4-5，流水控制模块位于第三级流水段，其主要功能是

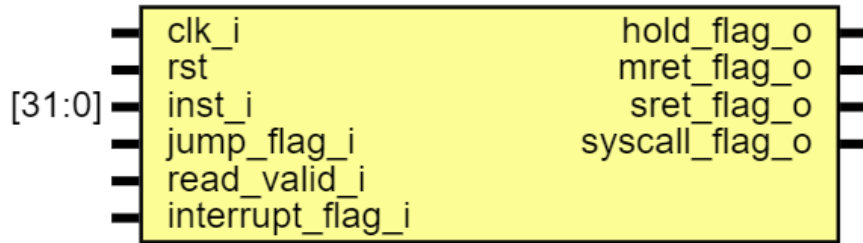


图 4-5 流水控制单元

控制 mret 信号与 sret 信号生成，同时处理流水线中的信号冲突问题，包括 jump 指令与中断指令产生的冲突。同时该模块也负责流水线停止信号的产生，当产生中断信号时会产生该信号。

具体的流水控制如图 4-6，store 与 load 指令将会占用地址出口，因此此处也需要做流水控制，图中标红处代表需要进行冲刷，即将该指令无效化处理。



图 4-6 三级流水控制

4.1.5 CSRS 与 REGS

RISC-V 的 CSR（Control and Status Registers，控制与状态寄存器）是处理器核心中

用于管理和监控系统状态的特权级寄存器，通过 12 位地址编码（如 0x300=mstatus）提供对中断控制、性能计数、调试跟踪等关键功能的配置。其核心功能包括：中断使能配置（如 mie 寄存器设置中断类型）、异常处理入口（mtvec 指向异常处理程序地址）、特权模式切换（mstatus 管理机器/监督/用户模式转换）、计时器控制（mtime/mtimecmp 实现时钟中断）以及自定义扩展支持（预留 0x7C0-0x7FF 地址供芯片厂商扩展）。通过 CSR 指令（如 csrrw、csrrs）或 ECALL/EBREAK 触发访问，结合特权架构（M/S/U 模式）实现硬件资源的安全隔离，是操作系统实现中断响应、上下文切换和资源调度的底层基石。

csr 寄存器组的索引有 12 个比特位，因此理论上 csr 可以容纳 4096 个寄存器，但是实际用不到这么多的寄存器，因此此处仅作声明，而不做具体使用，BLKRV 使用的 csr 寄存器包括 mtvec（存储中断查询程序）、mepc（存储中断时的 pc 值）、mcause（存储中断原因，这里处理为中断号）、mstatus（存储中断状态，此处仅使用了中断使能位）、satp（前 20 位存储虚拟页表基址，后 12 位存储进程号）、satp_s_cp（保存系统调用触发时的 satp）、satp_i_cp（保存中断触发时的 satp）、stvec（存储系统调用查询程序）。此处之所以设计了 satp_s_cp，satp_i_cp 是因为在系统中执行系统调用时也有可能触发中断。

REGS 的 32 个通用寄存器占用的索引位为 5，regs 部件也需要接收中断信号等，以防止在触发中断或者跳转需要冲刷流水线时错误写入寄存器。

4.1.6 中断处理

在中断处理模块中仅负责在中断使能时产生中断信号以及根据设备号生成中断标志。在流水中遇到中断信号时，处理流程如图 4-7。核心逻辑分为两部分：当中断信号触发时，硬件自动将 mtvec 基地址写入 PC 寄存器，同时保存 PC-4 至 mepc 并清空流水线以响应异常；当译码到 mret 指令时，流水线暂停并生成 mret_flag 信号，通过 PC-8 偏移规避转移指令冲突，最终将 PC 恢复为 mepc 存储的原始地址并清空流水线。图中特别用红/蓝色标注了关键路径，通过表格对比展示了正常指令流与异常处理场景下的 PC 更新策略，以及 mret 信号与跳转指令的互避机制，揭示了处理器异常返回的原子性处理缺失设计。

4.2 外设

4.2.1 外设虚拟化

BLKRV 系统中的外设设计采用 c++面向对象的开发模式。

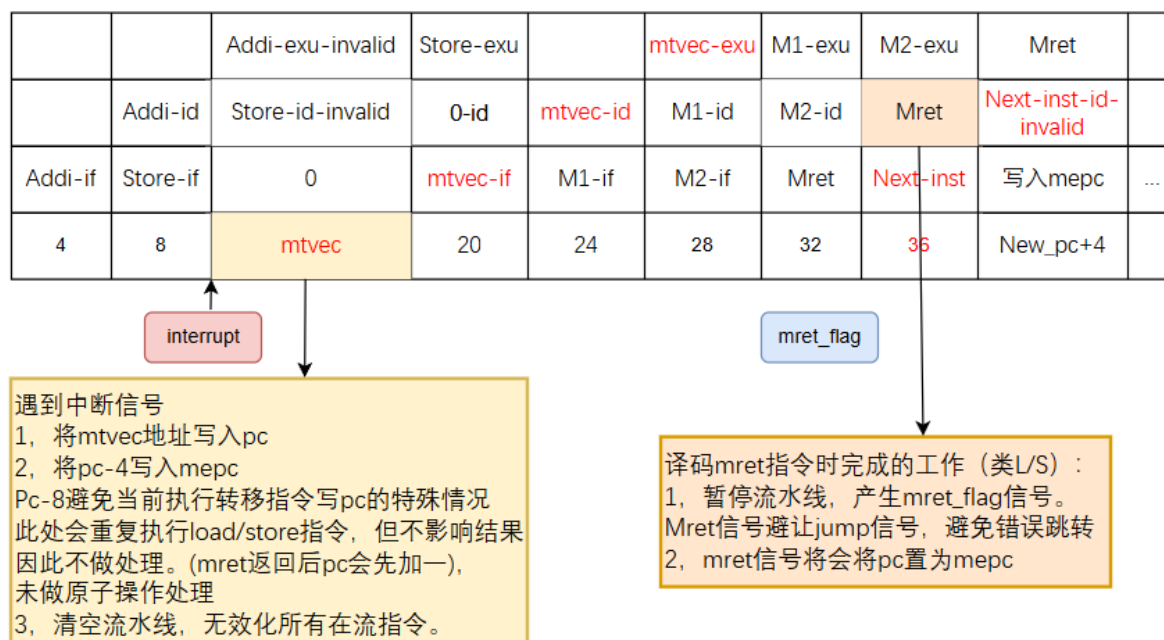


图 4-7 中断处理流程

首先需要知道的一点是，对于一个 32 的系统，其地址空间大小为 4GB，这是 CPU 能够访问的地址范围，而所有的外设即分布在这个地址空间范围，那么所谓的操作硬件，实际上也就变为操作存储器，当向某个地址写入内容时，如果该地址被定义为某个硬件的寄存器，那么此时的工作就是控制该硬件的寄存器。

理解了以上基本事实，那么就可以对所有的外设进行建模，可以认为所有的外设都属于存储器类型。ram 之类的易失性存储器其内容下电消失，因此将其放在堆上，而 flash 等非易失性存储器其内容需要永久保存，因此使用文件作为其本质。

基于以上，很容易抽象出整个外设群的联结关系，画出其继承关系类图如图 4-8。

所有的外设均是 mem 抽象类的子类，需要实现 process 方法，该方法是每个周期外设处理 top 信号的方法，所有外设都应该继承以便于发出中断或者传输数据。如果在 Wrapper 中调用外设，那么将会面临非常多的冗杂的外设细节，因此此处新定义一个 devices 类，将所有外设作为其类成员，对外提供统一的访问接口，其类图如图 4-9。

注意到 MMU, RIB-BUS 等设备也被虚拟化, 也是作为类出现在系统中, 因此这里额外添加关于 TOP, SOC, DEVICES 之间的类关系, 如图 4-10。

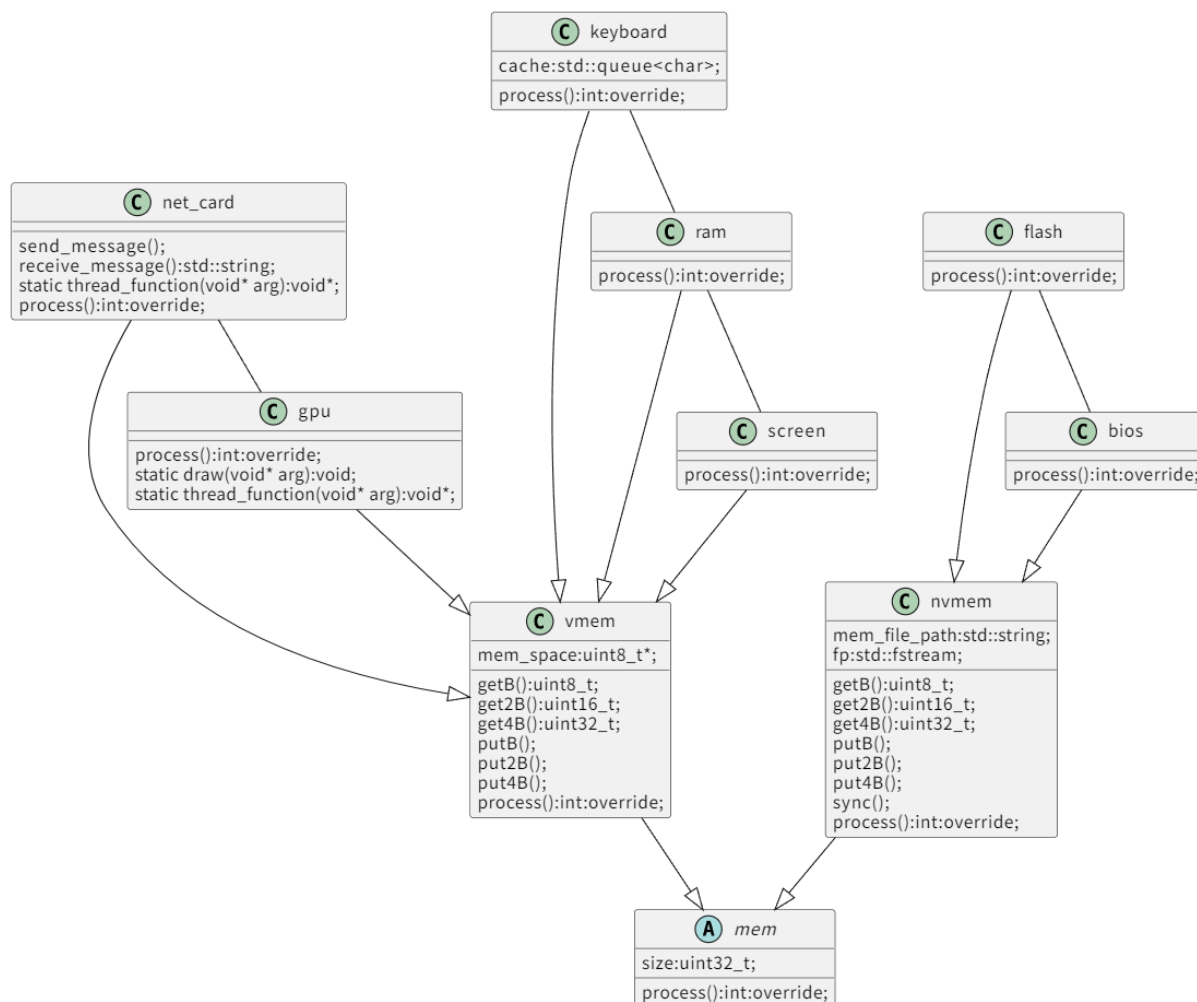


图 4-8 类图继承关系

总体的类间关系介绍完后, 需要对外设设备具体的实现进行一定的讲解, 出于性能考虑, BLKRV 并未考虑设备的延迟, 对于 **ram**、**flash** 等设备, 在物理上并不能在发送数据请求的周期拿到数据, 但是即使这么做也并不会影响系统所要表现的逻辑, 这里仅作为一个补充说明, 出于各方面的考虑对于 **flash**、**bios**、**ram** 等外设实现的是即时反馈, 即 TOP 发送了数据请求的那个周期, 设备就会将数据写入 TOP 模块, 从而拿到数据, 但是也会有意外情况, 例如对于网卡, 当基于计算机网络体系结构, 使用套接字与其他设备通信时, 再也无法控制其他设备的响应时间, 因此这里不得不将该设备的执行设计为一个独立的线程/进程, 这里实际为线程, 切换开销较小。

Devices 类作为硬件聚合容器，通过组合模式集成多种设备组件：包含 my_bios（启动固件）、my_ram（内存管理）、my_gpu（图形处理器）等成员对象，每个设备类（如

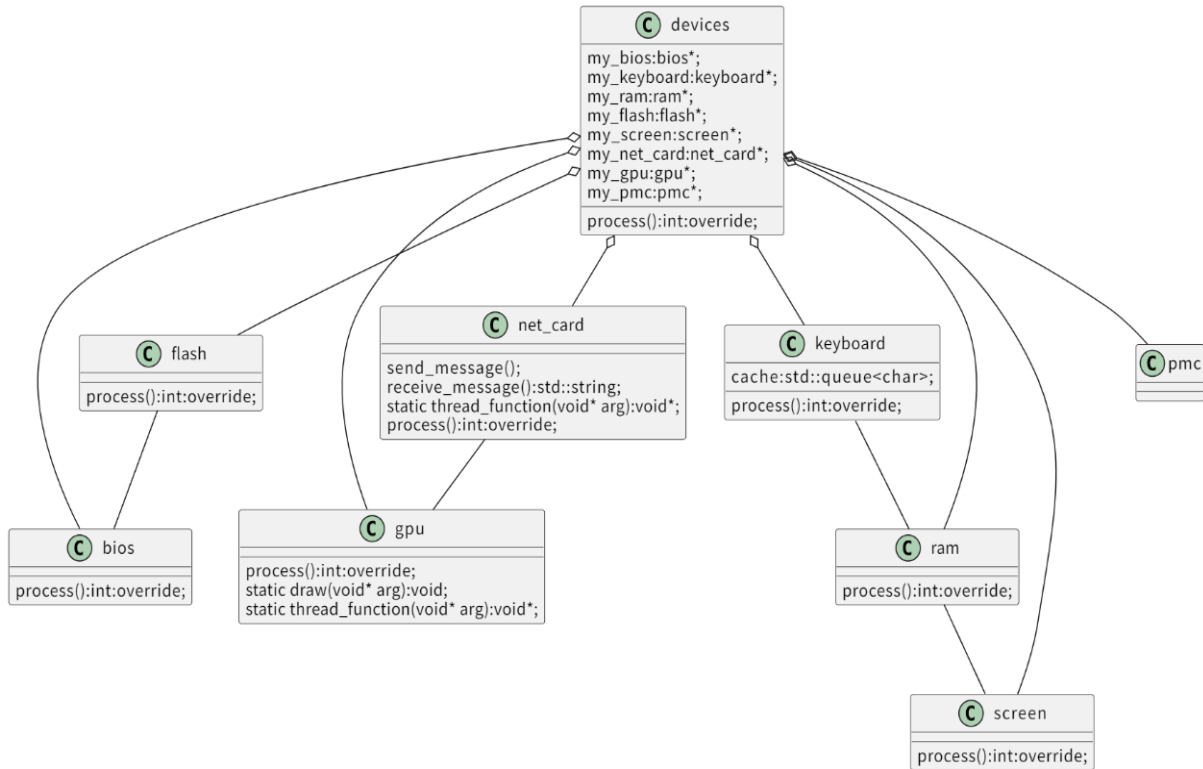


图 4-9 类图聚合关系

net_card、pmc）通过重写 process()方法实现多态行为，例如 gpu::draw()处理图形渲染，net_card::send_message()管理网络通信。关键设计特征包括：1）分层控制，Devices 统一调度硬件协同（如 pmc 电源管理单元包含缓存队列和线程函数）；2）异步机制，通过静态 thread_function 实现设备并行操作（如屏幕刷新）；3）标准化接口，所有设备通过 process()方法暴露生命周期管理逻辑。该架构以模块化方式模拟真实计算机的硬件交互，适用于教学场景中硬件行为可视化和系统级调试实验。

如果仍然让网卡设备在这里串行执行，那么需要面临的一个问题是，在进行网络通信的时段内，一个时钟周期将会包括网络连接，网络通信等等过程，这将会影响 BLKRv 的整体性能。同样的设备还有 GPU 设备等等。需要再次强调的是这里的 GPU 设备仅提供画图接口，而没有实现具体的渲染管线。具体的实现如伪。这里仅展示发送内容的处理部分。

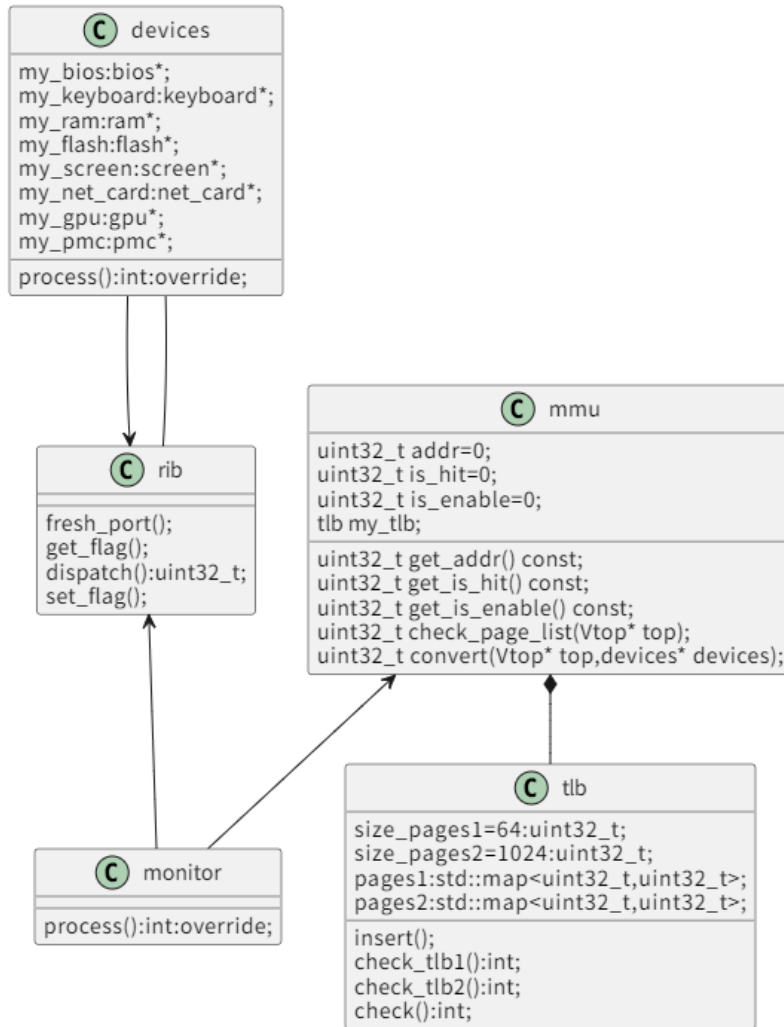


图 4-10 类图继承关系

代码 4-1 网卡处理独立线程伪代码

```

While do
    Get_sock()//阻塞获取从 os 传来的 sock 字段
    If get_dock do
        Get_content()//获取从 os 传来的内容
        Send()//向目标设备发送内容
        Send_flag()//向 os 传递结束 flag
    Endif
endwhile
    
```

4.2.2 存储器模型

需要注意的是实际的外设中,例如 RAM 随机访问存储器,与这里虚拟化的 RAM 存在着某些细节上的差异,要理解这一点,需要一个简单的门电路存储器,如图 4-11。当写入地址时,同时还需要传递取值类型,也就是字,半字还是一字节,如图 4-11 中的 mask。那么为什么需要一个 mask? 这里存在一个前提是存储的对齐访问模式,与想象

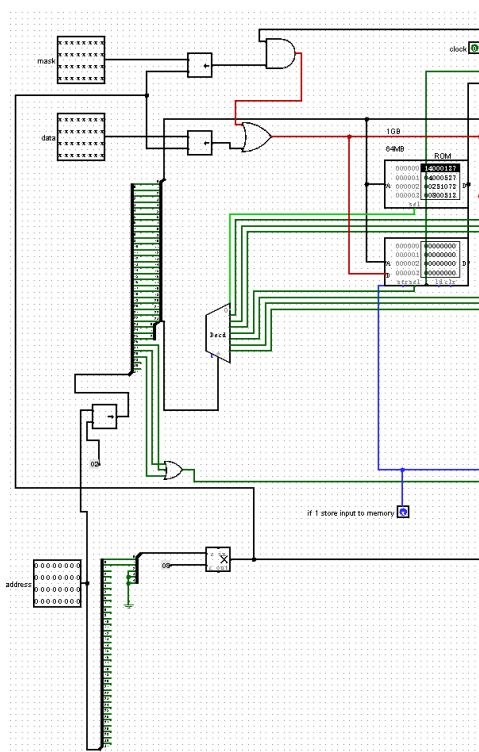


图 4-11 门电路实现的 RAM (Logisim)

的访问指定字节不同,实际一个四字节对齐的存储器,索引地址只能获取 4 倍字节的四字节数据,也叫内存对齐,因此借助 mask,可以掩藏掉不需要的数据,但是出现一个新的问题,就是如果访问的数据跨越两个四字节,那么此时存储器将花费多个周期去进行一个拼接操作,直觉上来看,很容易注意到这是个极耗时的操作,在虚拟设备中,为了性能,直接返回三个长度的数据,不再考虑内存对齐的问题。但是这里很容易添加该功能,如果需要验证,仅需要在 ram 设备中添加即可。

4.2.3 具体设备号及内存分布

前文提到,不同的外设分布在不同的地址上,如表 4-1。各个设备的空间都设置偏小,是因为考虑到宿主机的有限资源。

表 4-1 外设地址分布与端口

外设	端口	地址起始	设备内存大小
Bios	0	0x00000000	1MB
Ram	1	0x00100000	256MB
Keyboard	2	0x10100000	1MB
Screen	3	0x10200000	256MB
Gpu	4	0x20200000	256MB
Nic	5	0x30200000	256MB
Flash	6	0x40200000	512MB
Pmc	7	0x60200000	1MB
Monitor	8	0x60300000	1MB
Reserved		0x60400000	2.5GB

第 5 章 操作系统详细设计

5.1 子系统设计

操作系统在整个平台中的占比最大，作为连接硬件与软件的桥梁，操作系统几乎在为用户程序提供接口服务的同时，还需要处理好众多的硬件细节，因此这里将会花费较多的篇幅详细讲述操作系统的细节。以下统一将该系统称为 BLKos。

5.1.1 系统调用

首先需要明确的是用户程序调用系统调用的流程，以 `print` 函数举例，该函数实际调用的系统函数是 `vprint`，在 `linux` 中对应 `sys_write` 调用，`vprint` 函数的实际功能是调用 `ecall` 指令，实际并没有写入相应的打印的功能，实际的实现在 `vprint_i` 函数中，因此这里可以简单理解 `ecall` 实际上是一个跳转指令，当执行 `ecall` 时将会跳转到 `vprint_i` 对应的程序地址，从而完成系统调用。但实际过程比这要复杂一些，具体流程如图 5-1。

具体的流程为，当执行 `vprint` 时，`vprint` 函数首先将系统调用号写入 `a7` 寄存器，然

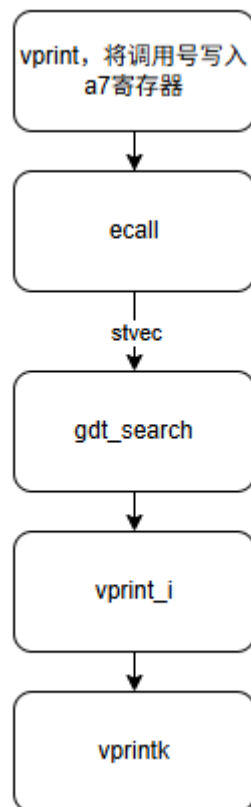


图 5-1 执行系统调用

后执行 `ecall` 指令，该指令将会直接跳转到存储在 `stvec` 这个 `csr` 寄存器的系统调用查询

程序地址，系统调用查询程序是一个汇编程序，如代码 5-1。系统调用声明如代码 5-2。

代码 5-1 系统调用查询程序

```
_gdt_search:
    slli a7,a7,2
    la a4,syscall_table//调用表基址
    add a7,a7,a4
    lw a7,0(a7)
    jalr ra,a7
    sret
```

代码 5-2 系统调用声明

```
#define _syscall2(type,name,atype,a,btype,b) \
type name(atype a,btype b) \
{ \
    type __res;\
    __asm__ volatile (\
        "li a7, %1\n"\
        "mv a0, %2\n"\
        "mv a1, %3\n"\
        "ecall\n"\
        "mv %0, a0\n"\
        : "=r"(__res)\
        : "i"(_NR_##name), "r"(a), "r"(b)\
        : "memory");\
    return (type) __res; \
}
```

该程序首先通过 a7 寄存器查询调用表，从而获取 vprint_i 的实际物理地址，从而跳转执行 vprint_i，紧接着 vprint_i 调用内核程序 vprintk，这里之所以需要区分 vprint_i 与 vprintk，是因为在内核中无法调用系统调用，因此需要增加一层抽象，让内核直接调用 vprintk 函数。在这个过程中需要保存上下文，但在 BLKRV 平台中，该工作由硬件自动完成，对于每个处理核而言，当前任务是唯一的，因此其保存的上下文为当前任务的上

下文。在 BLKRV 架构中，上下文仅仅包括除 a0 寄存器（返回值）以外的所有通用寄存器。当该工作被交给硬件后，理论上实际设备的性能将得到质的提升。

还需要注意到的一个问题是，内核态和用户态的切换包括了从虚拟地址到物理地址

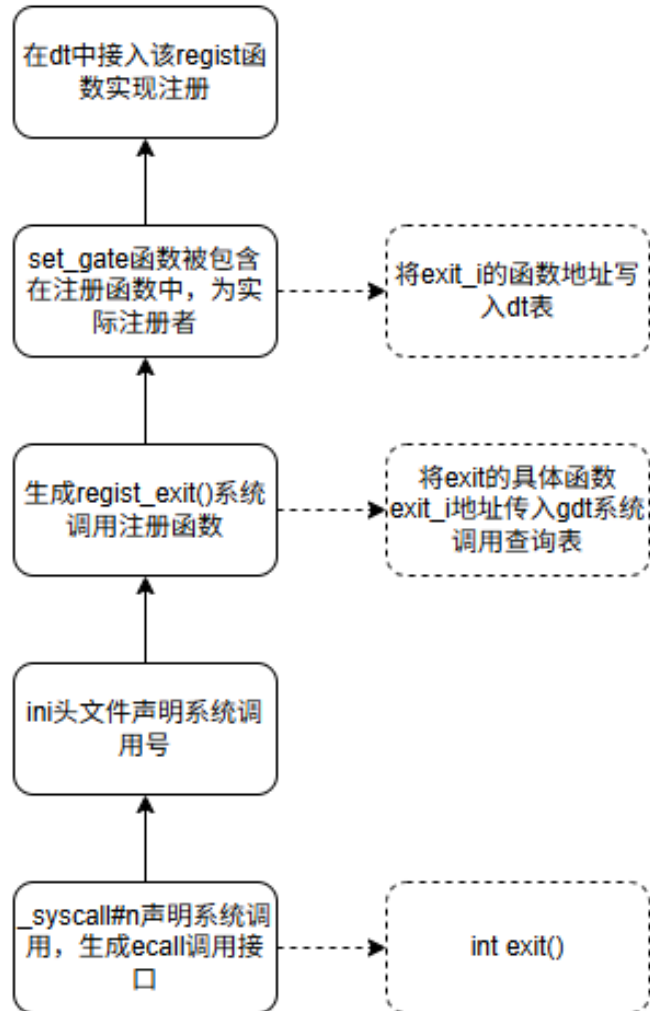


图 5-2 系统调用注册流程图

的转换问题，在 BLKRV 系统中仅有两种模式，机器模式和用户模式，前者运行在物理地址上，后者则是被隔离的虚拟地址，在执行系统调用时，ecall 指令或者中断同时还会触发 satp 归零，该寄存器不单负责虚拟地址转换，也用于控制 MMU 的开启与关闭，当其归 0 时，MMU 设备将被关闭，此时地址将直接从 CPU 发送到 RIB，也就是内核直接面对物理地址。实际上还有另一个解决方案是，将内核置于线性内核空间中，这样也能够达到一定的安全性，但是增加了抽象成本。还有个问题没有解决，那就是 a7 寄存器查询的地址从何而来，这就涉及到了系统中关于系统调用的注册，如图 5-2。

首先在 ini 中声明系统的调用号，从 0 开始，对于 n 号系统调用，存储其地址的调用表地址即为 syscall_table+n*4，regist_exit 函数由宏定义展开，宏定义如代码 5-3。

代码 5-3 系统调用声明

```
#define _regist_syscall(type,name) \
type regist_##name(int* dt_addr) \
{ \
    int* func_addr_##name=(int*)&name##_i;\
    _set_gate(dt_addr,func_addr_##name);\
}
```

Regist 函数调用 set_gate 将 vprint_i 的函数地址存入到一个四字节数组中，这个数组即是系统调用表。

5.1.2 文件系统

文件系统是操作系统用于管理和组织存储设备（如硬盘、SSD）中数据的核心机制，通过层次化目录结构和文件元数据（如文件名、权限、创建时间）实现数据的持久化存储与高效访问。其核心功能包括：1）逻辑抽象，将物理存储空间映射为文件/目录的树状结构；2）空间管理，通过块分配策略（如 FAT 表的链式分配、ext4 的 extent 连续分配）优化存储利用率；3）数据保护，借助日志（Journaling，如 NTFS、ext4）或写时复制（如 ZFS）防止断电导致的数据损坏。常见的文件系统类型包括 Windows 的 NTFS（支持 ACL 权限）、Linux 的 ext4（日志型）、macOS 的 APFS（快照与克隆），以及网络文件系统（如 NFS、SMB），它们通过统一的 VFS（虚拟文件系统）接口与操作系统交

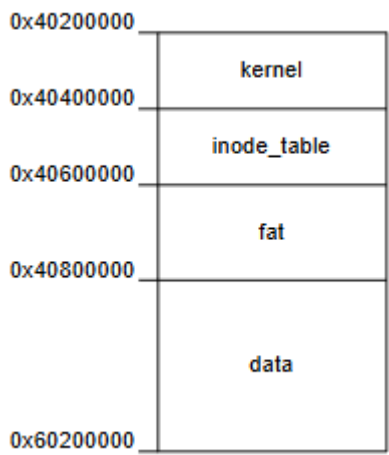


图 5-3 文件系统地址分布

互，确保应用程序无需关心底层存储差异即可实现跨平台文件操作。

对于文件系统，首先需要厘清的一个关键问题是，文件系统的所有相关信息都应该被永久存储的，即需要存储到 flash 设备当中去，不仅仅是文件的内容，还包括文件信息，块信息等等。这里采用的是经典的 FAT 文件系统，选择该方案的其中一个原因在于简单，第二个原因是 flash 设备较小，因此不会产生较大的 fat 表项占用，因此这里使

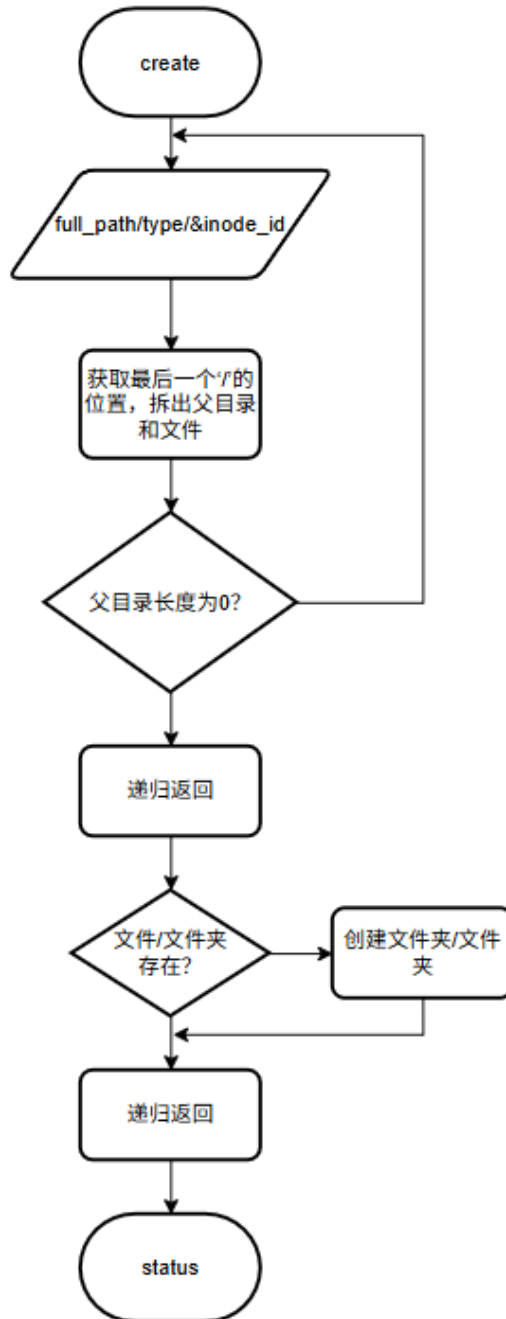


图 5-4 创建文件流程

用 fat 是其中一个较好的方案。在 flash 中规定的各项的地址分布如图 5-3。

前 2MB 地址用来存储操作系统内核，第 2-4MB 存储文件的 inode 信息，包括文件名，类型，开始 block，以及大小等信息，第 4-6MB 存储 fat 表，剩余内容存储数据。

接下来详细介绍文件系统提供的相关系统调用，首先是创建文件，其流程如图 5-4。当传入完整文件路径时，首先需要解析地址，这里采用递归的方式，逐步解析‘/’从而得到各级地址，当到达根目录时，根目录需要指定 inode_id，此时开始递归返回，每次查询该级解析出来的文件，如果不存在则创建，并返回文件 id 给上一级，从而每一级都

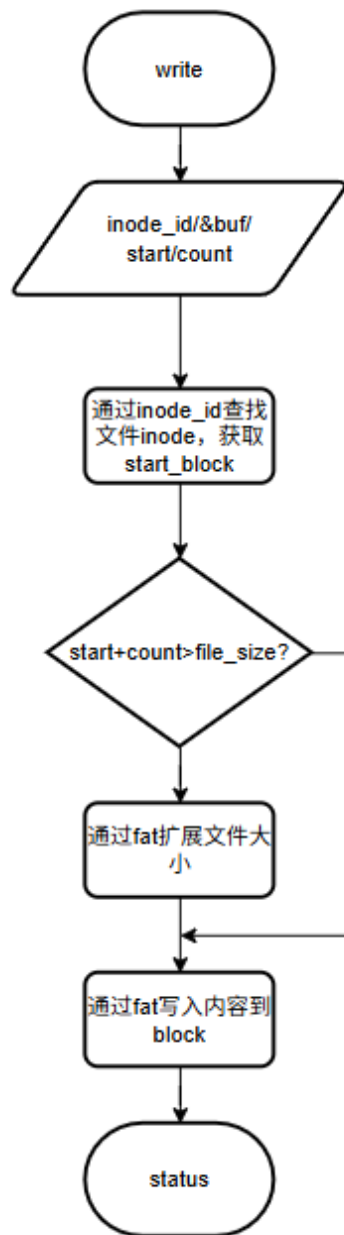


图 5-5 写文件流程

能查询并创建文件，并将文件写入到文件夹对应的 block 中去，这里特殊的是，常规的文件见 block 内容一般直接存储 inode，但是 BLKRV 存储的是 inode_id，这样可以节约空间的同时，写出更简洁的代码，缺点在于，只能创建硬链接，而无法实现软链接操作。

对于 open 系统调用，几乎与 create 调用的流程相同，只是在查询到文件，不进行文件的创建，而是直接返回文件的 inode_id。

对于 write 系统调用，其流程如图 5-5。区分文件的方式采用 inode_id，因此当用户程序想要操作一个文件时，前置条件是先调用 create/open，再进行 write/read 操作。此处 write 还需要传入 buf 指针，其存储需要写入的内容。通过 inode_id 查询到 inode 后，可以得到该文件的起始块，这个地方不再采用文件指针的方式，而是让用户传入 start 参数指定起始写入位置，这样可以避免掉繁琐的打开模式，但是同时带来一定不安全性。通过在 fat 表中不断的查询块，即可得到该文件的所有的块地址，从而实现写入操作，同时当查询块大小小于 start 时，采用直接查询下一块的方式提高 write 的性能。除此之外，write 还需要考虑的一点是，当写入内容大小超过文件本身大小时，如何扩展 block，即通过 fat 先分配 block，并产生链接，最后再索引到对应的 block 物理地址进行写入操作。

Read 系统调用与 write 实现几乎相同，只是缺少了扩展 fat 的步骤，同时将内容写入 buf。

5.1.3 内存管理系统

内存管理系统主要分为两个部分，第一个部分主要是利用 MMU 单元进行虚拟地址转换为物理地址，第二个部分是内存的管理。

对于虚址转换工作，这里需要补充说明 MMU 的具体工作，之所以放到这里才进行说明主要是为了内容的连贯性。需要知道的是，MMU 除了虚拟地址转换的功能，还增加了两级的 tlb 快表，当一个虚拟地址进入 MMU 时，首先会检查 tlb 的两级快表，如果命中，那么将会直接返回对应的物理地址，如果未命中，那么将会进行如图 5-6 所示的流程。

首先获取 satp 寄存器存储的页表基址，每一个进程都有自己的二级页表，第一级页表被称为页表目录，通过 satp 索引到页表目录，虚拟地址位[31:22]作为页表目录的偏移地址。

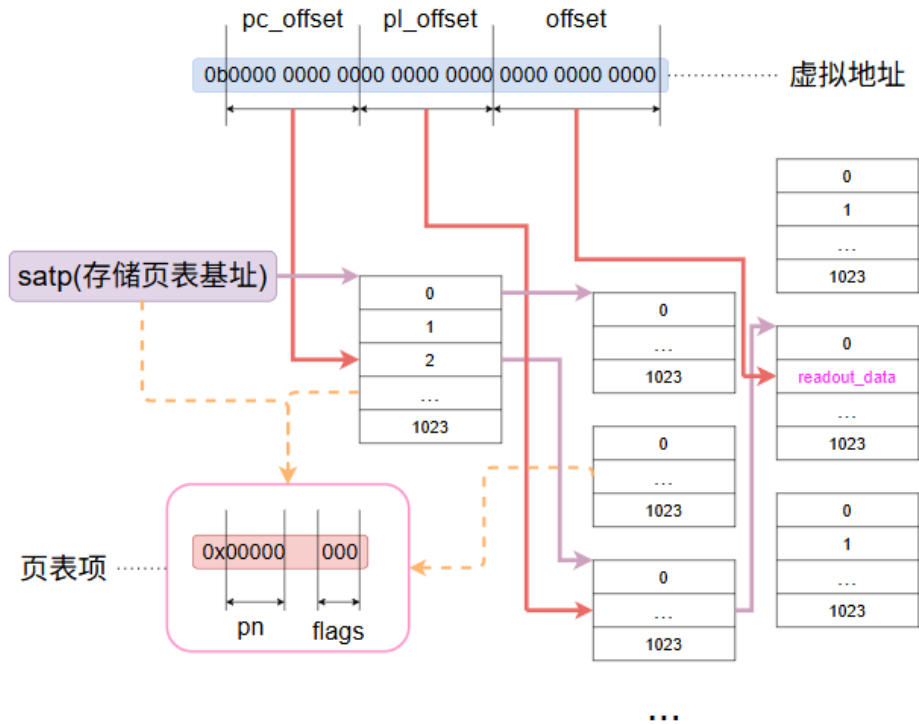


图 5-6 虚拟地址转换

比如当值为 20 时,意味着需要索引 `page_content_addr+20*4` 处地址内容拿到页表地址,接着解析出虚拟地址位[21:12]作为在页表中的偏移,即 `page_list+offset*4` 处的内容,此时拿到的是具体的物理页号,物理页号左移 12 位加上虚拟地址位[11:0]即可得到物理地址。

以上流程结束后,还需要将该地址写入到 `tlb` 快表,以便于下一次快速访问,同时 `tlb` 快表有一定的容量限制,这在系统中可以随意配置,此时如果 `tlb` 快表已满,就需要采用特定的换入换出策略,将旧的地址换出,新的地址换入,这里采用的是最近最少使用策略 (LRU),其核心思想是优先淘汰最久未被访问的数据。具体来说,系统会记录每个数据项的访问时间,当缓存容量不足时,选择最近最长时间未被使用的数据进行移除,从而为新数据腾出空间。该策略通过维护一个按访问时间排序的队列(通常用双向链表实现,配合哈希表快速定位),每次访问数据时将其移动到队列头部(表示最近使用),而淘汰时则从尾部删除最久未使用的数据。

LRU 广泛应用于缓存系统和页面置换算法,能够高效保留热点数据,提升系统性能。例如,浏览器的缓存机制或数据库查询缓存常采用 LRU 策略来平衡内存使用与访问效率。



图 5-7 内存管理图例

关于第二个部分内存管理，其主要的系统调用是 `malloc` 以及 `free`，虽然采用了虚拟地址转换，但是分配的虚拟地址也需要是连续地址，因此管理内存的方式尤为重要，如果仅仅是每次分配一个物理页，那将会面临即使只需要四字节数据，也分配 4096 字节的情况，因此这里采用空闲链表的方式管理内存，新的问题在于，链表节点的初始化依赖于 `malloc`，而 `malloc` 又依赖于链表，产生了互相依赖，因此此处的链表不是常规意义上的链表，BLKRV 将每个节点直接存储在内存中，使用地址指针的方式去访问 `next` 指针。如图 5-7。

Pcb 中存储了头指针，用来指向内存中的第一个空闲块，这里之所以没有将头指针放到内存开始，是因为 `malloc` 的实现方式中，`free node` 将会被 `used node` 直接替换，如果头指针被替换将会出现无法找到空闲链表的错误。BLKos 的内存管理中没有考虑最佳的内存块，当找到大于分配空间的第一个空闲块即分配。

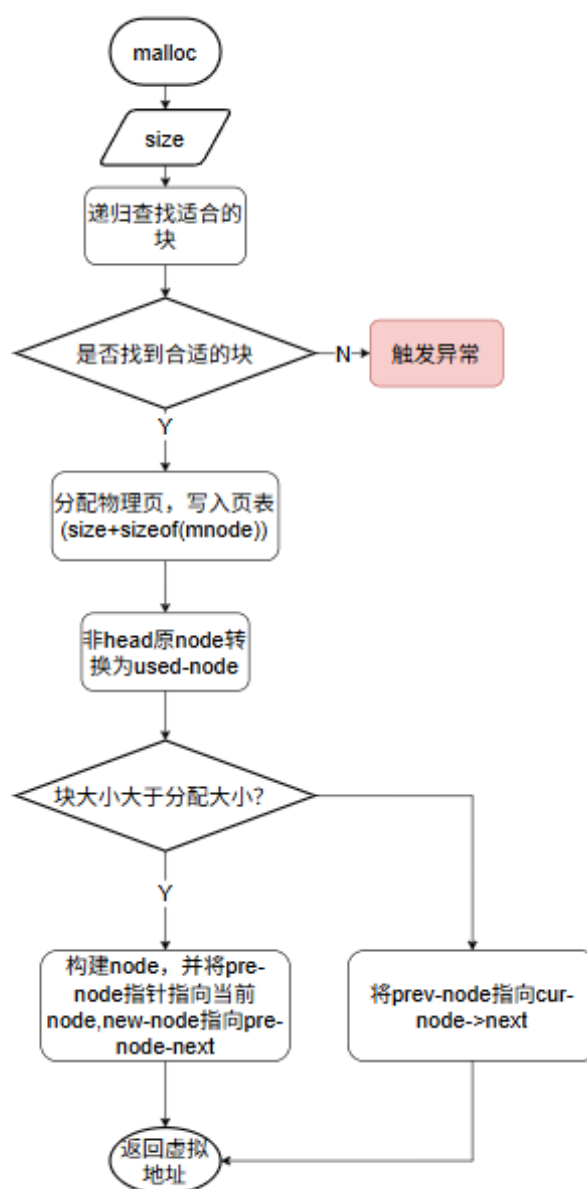


图 5-8 malloc 流程图

注意到 free node 中存储的是 vir_next, 也就是虚拟地址, 需要经过代码转换为物理地址才能够访问 node, used node 将不会出现在链表中, 只有 free 时才会被替换为 free node, 也就是在常规状态下, free node 与 used node 都是相互转化的, 这样降低实现的复杂度。Malloc 的流程如图 5-8。

在 BLKos 中是实时分配物理页的, 并没有像常规的内存管理系统, 在进程启动时即分配了一定的堆空间, 我们在用户需要时才进行物理页的分配。这样做可以达到节省空间, 缩短进程拉起的时间损耗的目的。

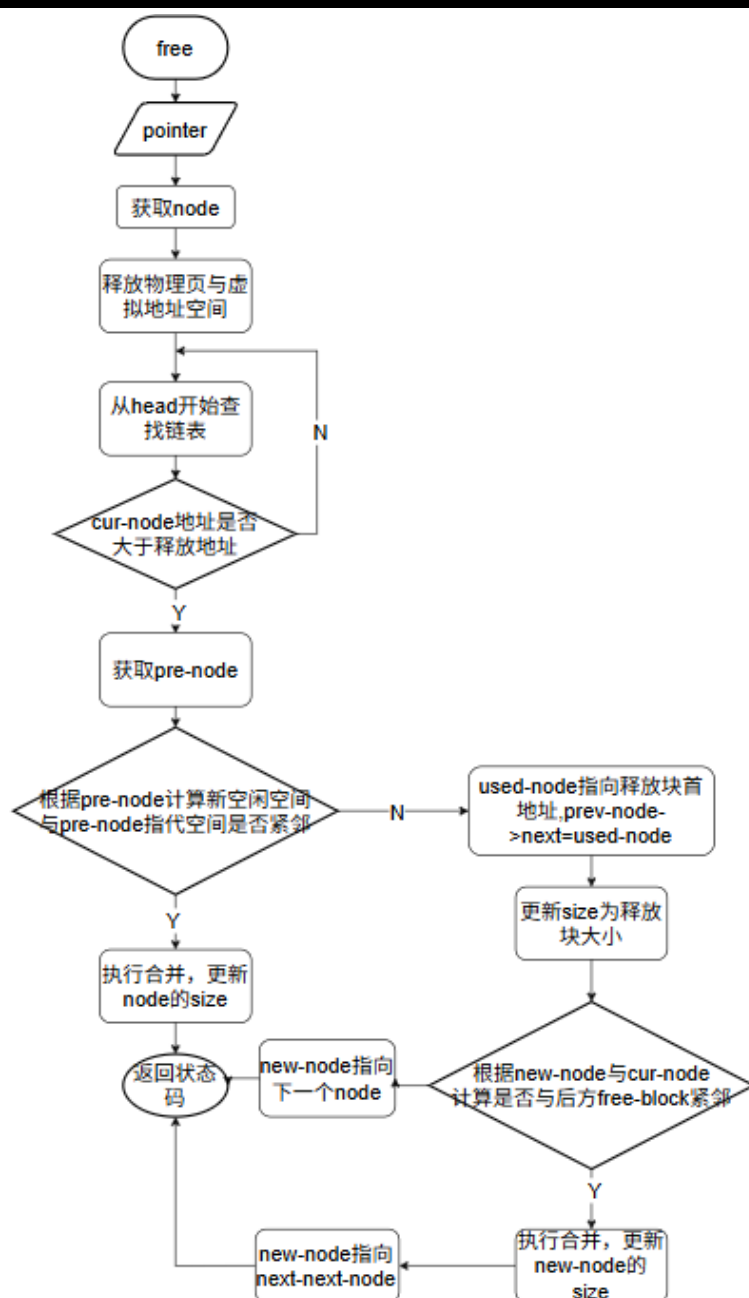


图 5-9 free 流程图

以上过程清晰明了, 唯一需要注意的点是, 当分配的内存大小小于空闲块大小时, 将会产生新的空闲节点, 此时也需要重构链表, 需要注意链表中虚拟地址与物理地址相互转化的问题。Free 的流程如图 5-9。

Free 流程较为复杂, 因为涉及到前向合并与后向合并问题, 如图 5-9 中所示, 当释放块前/后存在紧邻的空闲块时, 需要进行内存合并, 以避免出现内存碎片问题, 而造成即使有空闲空间也无法分配的情况。

内存管理的内容基本结束，还需要注意的是，并不是一整个内存都是可分配的，内核将会使用一部分内存用来执行内核程序，同时在系统中系统调用实际执行栈位于这一块内存当中，也就是前 4MB 大小的地址，前 2MB 用以装载内核程序，紧接着是系统调用栈，中断调用栈，以及内核栈。

5.1.4 I/O 子系统

对于输出，实际的应用在于 `print` 这一层，即内容的打印，前面对于 `screen` 的驱动已经有了详细的讲解，因此不过多进行赘述，除此之外，需要讲的是重定向与缓存，当调用 `vprint` 时需要传入一个 `buf`，这个 `buf` 是需要输出的内容，由此可知，BLKRV 输出缓存实际上由调用 `print` 侧管理，在 BLKos 中，在哪里实现了 `print`，就需要在哪里设置缓存，这是由于跨文件访问变量时，由于 `ram` 起始地址在 `bios` 之后，不是从 0 开始，因此 `got` 表无法获得真实的地址，因此需要在每个实际实现的地方增加一个 `out_cache` 缓存。在实际应用时，用户程序编译需要包含 `std` 文件，其中实现了 `print` 并且预设了一个输出缓存。

对于输入系统，目前仅仅设计了键盘输入，当键盘按下时，会产生一个中断，中断的管理与系统调用大同小异，事实上，两者都属于“程序陷入”。此时进行键盘的中断程序处理，BLKos 读取键盘缓冲区数据，接着存入设计好的环形缓冲区内，并更新环形缓冲区的读写指针，环形缓冲区（Circular Buffer 或 Ring Buffer）是一种高效的数据结构，常用于生产者-消费者模型的场景（如数据传输、音视频流处理）。其核心是一个固定大小的连续内存空间，通过两个指针（或索引）——读指针和写指针——循环覆盖旧数据，实现数据的先进先出（FIFO）管理。当指针到达缓冲区末尾时，自动绕回到起始位置，形成逻辑上的“环形”。无锁（Lock-Free）设计是环形缓冲区在高并发场景下的关键优势，其核心思想是通过原子操作或内存屏障避免传统锁机制的开销，从而提升性能。虽然此处并没有事实意义上的多线程，但是多进程中，该环形缓冲区也同样避免了锁的出现。

5.1.5 进程管理

这里不讨论算法形式上的进程管理，事实上大多数教材仅仅提到进程管理中的 `scheduler` 部分，即如何安排哪一个进程执行这个问题，但是对于进程上下文，进程的调用过程等等置若罔闻，导致初学者学完后仍然一头雾水。

这里将会详细讲解其完整的流程。对于 `exec` 调用，其基本覆盖了整个进程管理的周

期（这里仅讨论单核），当使用 `exec` 系统调用拉起一个文件程序时，首先需要初始化 `pcb`，这里包括进程号，进程上下文，进程状态，等等一系列信息，出于简化考虑并没有放入太多信息，仅使用了几种必要的。

然后需要初始化页表，页表的作用是提供虚拟地址与物理地址建立映射的桥梁，前文详细提到了虚拟地址转换的过程，因此进程的页表初始化是一个很重要的过程，在内存管理中，空闲块链表的建立也是基于页表的连续地址，可以这样认为，页表几乎就是进程的内存空间。

接着需要进行程序的加载，这是很容易被忽略的过程，如果查看二进制文件，会发现二进制文件格式中存在 `program header`，如图 5-10，该段记录了哪些内容需要被加载，以及该内容的权限，文件大小，占用空间大小等等信息，操作系统根据这些信息去加载二进制文件，获取程序入口，除此之外，操作系统还需要将传入的参数复制到用户空间以便于用户程序使用。

Program Headers:							
Type	Offset	VirtAddr	PhysAddr	FileSiz	MemSiz	Flg	Align
RISCV_ATTRIBUTES	0x002ee0	0x00000000	0x00000000	0x0001c	0x00000	R	0x1
LOAD	0x000000	0x00010000	0x00010000	0x02ed0	0x02ed0	R E	0x1000
LOAD	0x000000	0x00013000	0x00013000	0x00000	0x00408	RW	0x1000
GNU_STACK	0x000000	0x00000000	0x00000000	0x00000	0x00000	RW	0x10

图 5-10 program header

最后，通过汇编原子指令，设置上下文，如代码 5-4。

代码 5-4 上下文设置

```
__asm__ volatile(
    "mv t0,%3\n"//加载程序页表基址
    "mv t1,%4\n"//加载程序入口地址
    "mv a0,%0\n"//argc
    "mv a1,%1\n"//argv
    "mv sp,%2\n"//栈底
    "csrw satp,t0\n"
    "jalr ra,t1\n"
    "nop\n"
    "nop\n"
```

```
::"r"(para_num),"r"(argv),"r"(stack_bottom),"r"(global_pcb_list[new_pid].satp),"r"(prog_start_addr)); //设置 satp
```

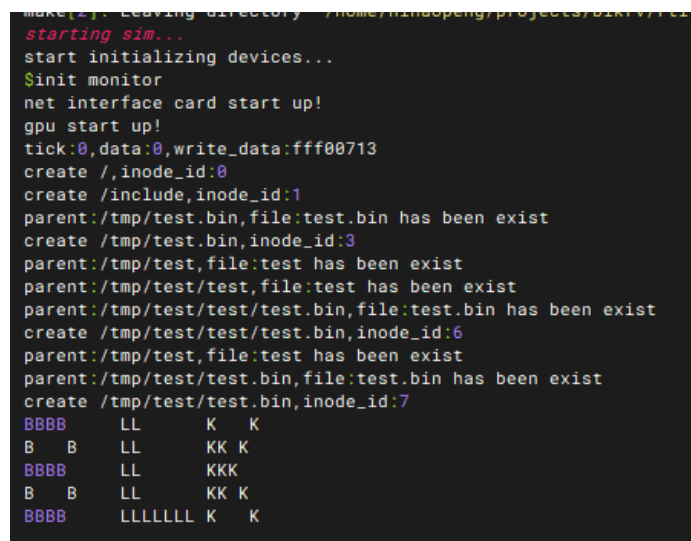
以上之所以需要使用临时寄存器，因为直接使用变量的寄存器会产生冲突，最后跳转到程序入口，对于用户的程序入口，由于自定义了程序架构，因此这里需要自定义一个 `_start` 入口函数，如代码 5-5。

代码 5-5 用户程序入口

```
.text
.global _start
_start:
    addi sp,sp,-12
    sw ra,4(sp)
    sw s0,8(sp)
    addi s0,sp,12//以上为栈帧操作，进入_start之前，sp应被保存然后修改
    jal ra,main//a0-7 默认转存不进行入栈操作，
    li a7,9
    ecall
```

当从 `main` 程序返回后，需要程序主动调用 `exit` 系统函数，该函数会执行包括资源回收，进程号回收，调用 `scheduler` 决定下一个进程等等一系列工作。

5.2 效果图演示



```
make[2]: Leaving directory /home/minaopeng/projects/blkv7/v7.1
starting sim...
start initializing devices...
$init monitor
net interface card start up!
gpu start up!
tick:0,data:0,write_data:fff00713
create /,inode_id:0
create /include,inode_id:1
parent:/tmp/test.bin,file:test.bin has been exist
create /tmp/test.bin,inode_id:3
parent:/tmp/test,file:test has been exist
parent:/tmp/test/test,file:test has been exist
parent:/tmp/test/test/test.bin,file:test.bin has been exist
create /tmp/test/test/test.bin,inode_id:6
parent:/tmp/test,file:test has been exist
parent:/tmp/test/test.bin,file:test.bin has been exist
create /tmp/test/test.bin,inode_id:7
BBBB LL K K
B B LL KK K
BBBB LL KKK
B B LL KK K
BBBB LLLLLL K K
```

图 5-11 输出“BLK”

此处将进行部分系统调用效果展示，如图 5-11，输出“BLK”。



图 5-12 绘制 BLKRV

如图 5-12，通过 GPU 绘制“BLKRV”。该界面通过 x11-server 显示到 windows 上。

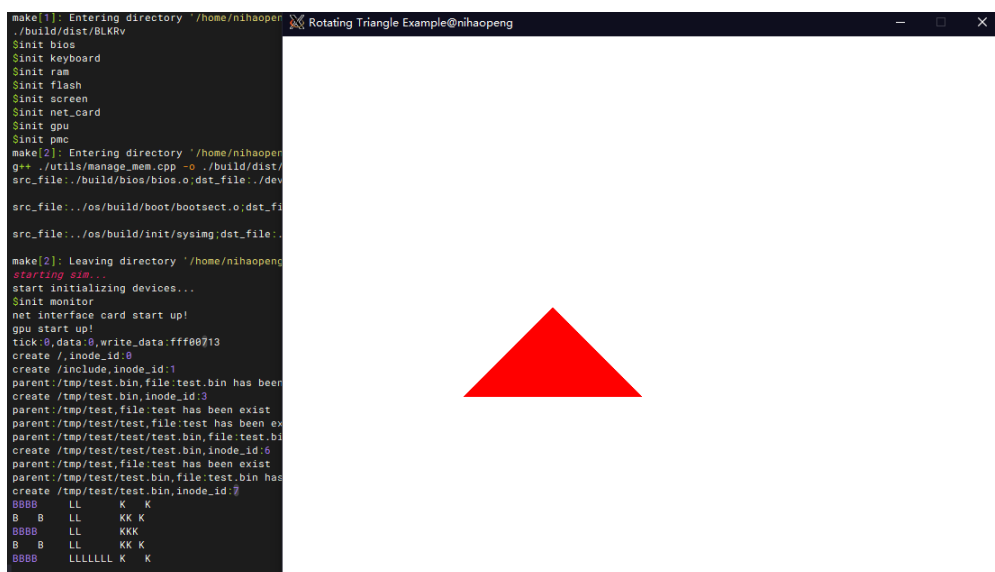


图 5-13 绘制三角形

如图 5-13，通过 GPU 绘制三角形。如图 5-14，用户贪吃蛇程序，借助基础的绘图功能实现。



图 5-14 用户程序“贪吃蛇”

第 6 章 模块等价替换

本章将通过文件系统这个例子详细讲解如何简单更换 BLKR_v 中的一个模块以验证算法。

6.1 文件系统等价替换实例

如果用户希望能够替换文件系统，使用自己设计的文件系统，同时测试文件系统的相关性能，首先需要设计文件系统的基本规则，这里采用 block 链表作为新的文件管理方式，如图 6-1。

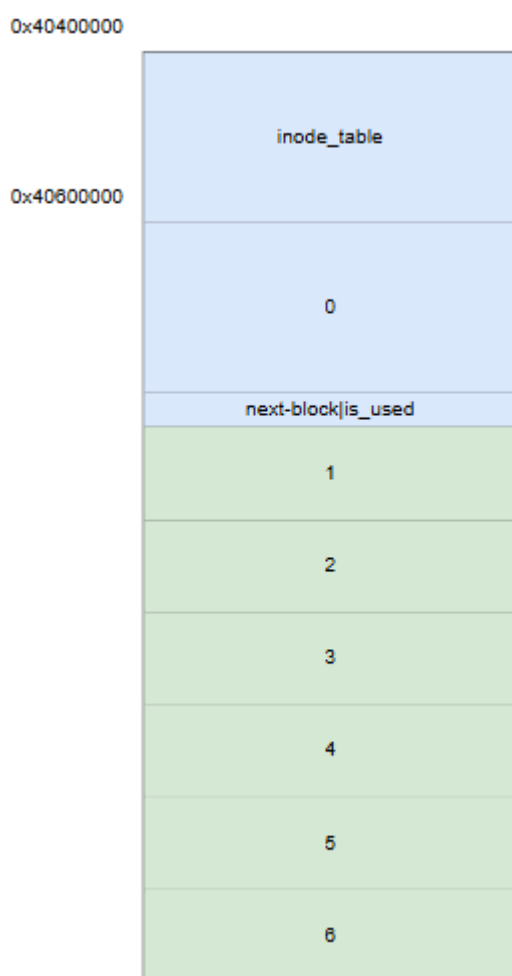


图 6-1 block-链表

该方式中，每一个 block 的尾部都被替换为 node 节点，例如对于一个 4096 字节的 block，第 4092 地址处的四字节作为 node，该 node 含有两个信息，下一个 block 的 id，以及当前块是否被使用。通过以上方式能够通过 inode 的 start_block 信息找到文件的所

有 block,从而达到访问数据的目的。在图 6-1 中还能看到数据的分布方式,例如 0x40400000 为 inode 数据存放起始地址,0x40600000 为数据存放起始地址,这种文件管理方式相比于 FAT 表管理文件的优点是,不需要单独开辟一块空间用来存放 fat 表,同时其性能并不受影响,但是在代码复杂度维度上,可能将会有不小的提升,因此在当前的系统中仍然采用的是 FAT 这种实现方式。

在确定好规则后,文件系统向上提供的能力即是提供访问文件的能力,因此需要 1,实现相应的系统调用;2,完成系统调用的注册。以上两个步骤即可完成整个模块的替换。对于第二点,仅需要遵照 4.3.1 节的流程完成注册即可,对于第一点,如果需要一个获取文件信息的系统调用,那么需要提供至少三个函数,内核函数,调用函数以及接口函数,对其定义分别为, `get_info_k`、`get_info_i`、`get_info`, `get_info_k` 函数用于实现具体的获取文件信息的功能, `get_info_i` 用于调用 `get_info_k` 函数,该函数的实际地址将被注册到 `gdt` 表中,而 `get_info` 函数则是向用户程序提供的接口,所有的用户程序中能且仅能访问该接口,同时该函数应该声明在 `std` 库中,用户程序编译时应该包含该函数的实现,如 `_syscall1(int,open,const char*,file_path);`

完成以上工作后,首先需要做的是更新用户程序的 `std`,这里体现出的是用户程序与操作系统之间的隔离,同时也看到了计算机体系结构分层的伟大之处,即使 `std` 库与 `os` 提供的函数不同,这里也仅仅会影响应用程序本身,而不会对其他程序产生任何影响。

最后,需要设计足够详尽的测试用例以验证算法的有效性,在这之前,首先需要重编译整个内核,然后重新加载内核程序到 `flash` 中,这里不必担心, `BLKRV` 提供了足够完善的工具来进行内核加载工作,你可以理解为烧录工具,而使用它的方法仅需要一个小小的 `make` 指令即可。当执行 `make` 时,实际执行的功能包括加载 `bios` 固件,将 `bootsect` 加载到 `flash` 前 512B,将内核 ELF 文件加载到 `flash` 指定起始位置,随后启动机器,开始执行程序。

除了以上模块替换流程,还有必要讲解整个系统的拉起流程以便于开发者了解具体的细节。如图 6-2。执行 `make` 时首先会进行工具的编译,这些工具可以理解为烧录工具,接着执行 `bios` 程序的编译,该程序的作用是拉起操作系统,更准确的描述是,拉起床唤醒操作系统的程序,也就是后面编译的 `bootsect` 程序, `bios` 的功能非常简单,仅仅是将 `flash` 设备前 512B 的数据加载到 `RAM` 设备中,并跳转到起始地址执行 `bootsect`,编译完 `bootsect` 后,工具会将该程序写入到 `flash` 前 512B 的位置,紧接着将 `os` 内核进行编译,并加载到 `flash` 512B 以后, `bootsect` 程序将会读取 `os` 内核的 ELF 文件,确认需要

加载的程序头, 并按照其大小以及物理位置进行程序的加载, 加载完成后将会跳转到 os 内核的入口函数, 也即 `_start`, 随后 os 内核执行初始化工作, 包括文件系统初始化, 虚拟内存初始化等等, 当然也包括注册系统调用的流程, 即 `regist_sysmethod`, 初始化结束后, 将会拉起 shell 终端, 这也是用户最常接触的对象, 容易让用户误解其为操作系统本身, 实际上 shell 仅仅是个常规的应用程序。

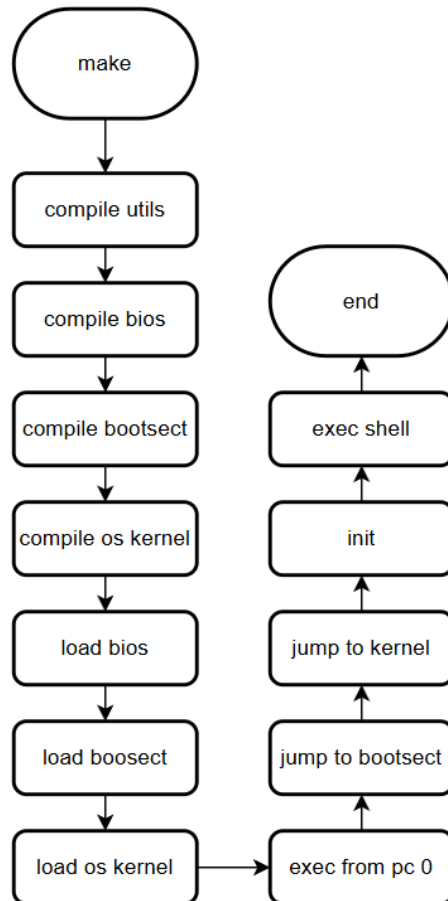


图 6-2 工程引导

6.2 CPU 核替换

计算机体系结构中, CPU 通过指令集架构 (ISA) 向上层软件 (操作系统、应用程序) 提供标准化的计算功能接口, 其核心服务包括: 1) 运算执行 (算术逻辑操作)、2) 内存访问 (加载/存储指令)、3) 异常处理 (中断与陷阱)、4) 特权级控制 (用户态/内核态切换)。这种隔离性通过硬件抽象层 (HAL) 实现——ISA 作为软件与硬件的“契约”, 确保无论 CPU 采用何种微架构设计 (如流水线深度、分支预测策略、缓存层级), 只要严格遵循 ISA 规范 (如 RISC-V RV32I、x86-64), 上层软件的行为均保持一致。例

如, Intel 和 AMD 的 x86 处理器虽内部实现迥异(如 Intel 的 Sunny Cove 与 AMD 的 Zen3 微架构), 但均兼容相同指令集, 可运行完全一致的 Windows/Linux 系统。这种分层设计通过编译器(将高级语言映射至 ISA)、操作系统(基于 ISA 实现系统调用)的双重抽象, 使得 CPU 制造工艺升级(如 7nm 至 3nm 制程)或架构优化(如乱序执行改为顺序执行)对软件完全透明, 确保了计算生态的长期兼容性与可扩展性。

在 BLKRV 中, CPU 通过 Vtop 接口向上层模块提供指令执行能力, 因此在系统中, 能够非常轻松地替换 CPU 核, 而无需考虑整个系统的具体细节, 这里以 QEMU 等价替换 verilog 实现的 CPU 核为例。

在 BLKRV 系统中将 Verilog 实现的 CPU 替换为 QEMU 模拟器, 需通过以下五层抽象实现无缝切换:

1. 接口标准化

定义统一的 CPU 行为接口(如指令执行、寄存器访问、中断响应), 使上层操作系统无法感知底层是硬件流水线还是软件模拟器。QEMU 需封装为符合该接口的模块, 对外暴露与 Verilog 核完全一致的控制函数。

2. 内存与中断桥接

将原 Verilog 核的物理内存访问重定向至 QEMU 虚拟内存系统, 通过回调机制实现 BLKRV 设备模型与 QEMU 地址空间的交互。中断控制器信号需转换为 QEMU 支持的虚拟中断注入协议(如 virtio-IRQ), 保持原有中断优先级逻辑。

3. 启动流程适配

改造 BIOS 引导机制, 使 QEMU 能直接加载 BLKRV 操作系统的 ELF 镜像。原 Verilog 核的 Flash 烧录流程转换为 QEMU 的 -kernel 参数加载, Bootloader 跳转地址需对齐 QEMU 虚拟机的默认入口点(如 0x80000000)。

4. 验证与调试优化

利用 QEMU 内置的 GDB 调试接口实现指令级单步追踪, 替代 Verilog 仿真的波形调试。功能性测试完全复用原有测试用例(如系统调用、内存管理), 性能评估指标从时钟周期数改为实际执行时间, 凸显模拟器快速迭代的优势。

这种替换验证了计算机体系结构的分层抽象原则——只要严格遵循 ISA 接口规范, 硬件实现(Verilog 物理核/QEMU 功能模型)的差异对操作系统完全透明。教学场景中, 学生可快速切换两种模式: QEMU 用于快速验证系统功能, Verilog 仿真用于观察流水线等硬件细节, 形成“功能-实现”的双视角认知。

第 7 章 可视化工具设计

7.1 monitor 设备

BLKRV 还设计了一个特别的硬件,即监测设备,该设备与 RIB-BUS、MMU 同级,位于 TOP 之上,同时收集他们三者的运行数据,由于该设备依赖于总线和 MMU 的类,因此如果仍然将之当作普通的设备看待,这里会存在引用冲突,所以将它从 `devices` 类中隔离出来单独执行,为了收集 TOP 的数据, TOP 应该在运行过程中向外暴露一些参数, BLKRV 将一部分的指令类型集合变成了一个值进行输出,即 `inst_type={22'd0,lui,auipc,jal,jalr,bj,load,store,calc,calci,sys}`; 剩余 22 位保留,在每一个运行周期,都会将 `inst_type` 变量返回到上层, `monitor` 设备将解析该数据进行数据留存。Monitor 设备同时会将各类型的指令执行以时间戳加是否执行的方式进行存储,一边绘制相应的流水图例。

同时 `monitor` 设备会收集 RIB 中对各设备的访问频率,以及 MMU 中 `tlb` 的命中次数等等,同时在 BLKos 中加入了两个系统调用,即 `open_monitor` 以及 `close_monitor`,前者可以启动 `monitor` 设备开始监测,后者可以关闭已经打开的 `monitor` 设备,单独执行时无效,当执行 `close` 时,所有的监测数据将会被自动写入文件,由于存在这两个系统调用,且他们皆是原子化的,因此这里能够在一段代码前后执行,已收集这一段代码的执行数据,这可以帮助开发者去改善代码的质量。

7.2 数据可视化分析

当然 BLKRV 也为 `monitor` 得到的监测数据设计了相应的可视化工具 BLKvis,通过使用 `chart.js` 绘制相应的可视化图像。对于程序如代码 7-1。

代码 7-1 测试代码段

```
for(int i=0;i<10;i++){
    char* tmp=(char*)malloc(30);
    str_cpy("this is blkrv malloc test",tmp);
    printf("tmp:%d:%s\n",(uint32_t)tmp,tmp);
    free(tmp);
}
```

代码 7-1 循环十次,单次循环分配 30 字节的空间,这在我们的内存管理系统中将会产生内存非对齐操作,将字符串复制到 `tmp` 变量当中以触发 LOAD/STORE 指令,打

印是否执行成功，最后释放该指针。



图 7-1 可视化界面

此时在代码的上下添加开启/关闭 monitor 的接口，执行该程序即可在工程目录得到 data.txt 文件，在 BLKvis 启动一个 http 服务，这个服务可以使用任何支持 http 访问的工具，这里使用最简洁最方便的 python 模块 http.server 直接启动工程。得到可视化数据指令执行时间如图 7-1。可以发现 LOAD/STORE 指令几乎占了绝大部分，因此可以认为程序的主要瓶颈实际在内存访问，实际上，大部分主机程序的瓶颈都在访存，因此实际工作中做了大量的相关改进，例如三级缓存等。

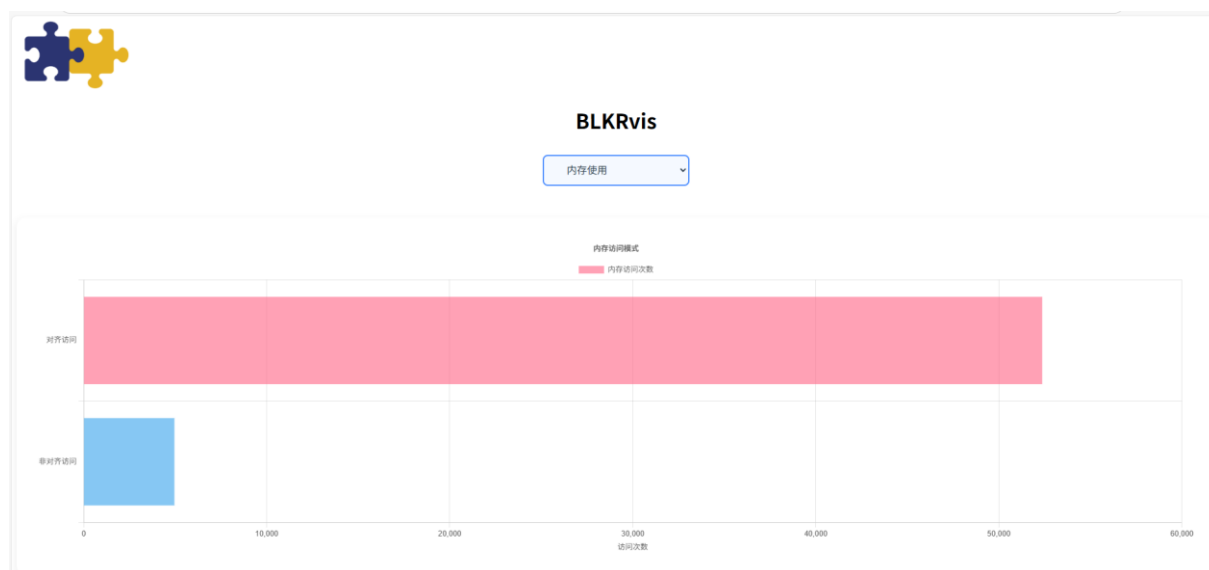
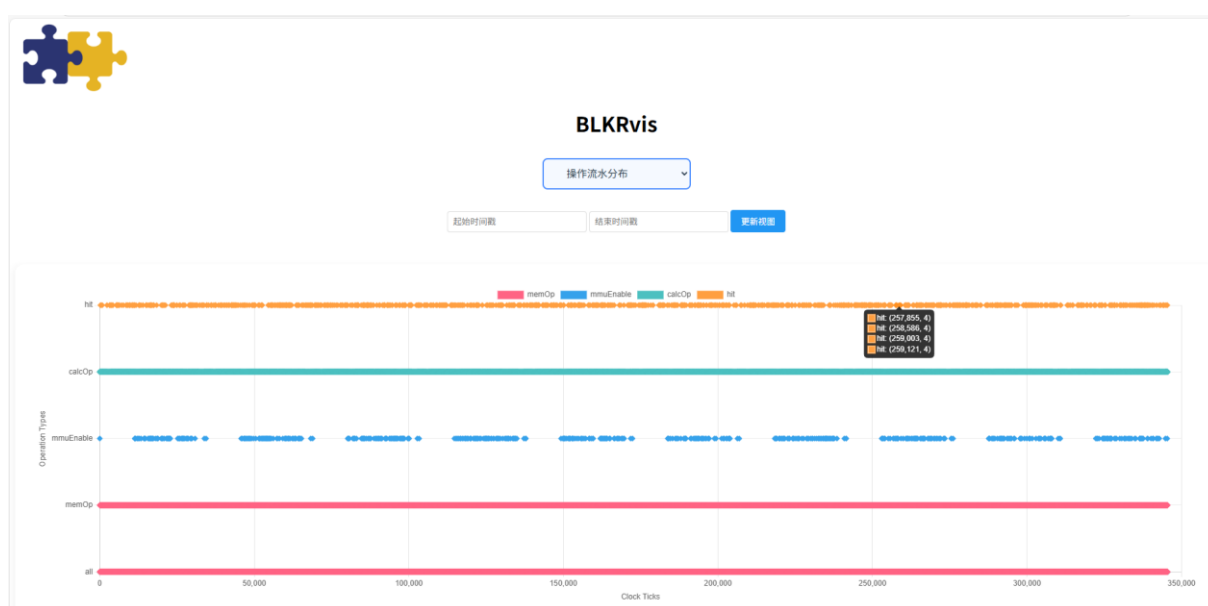


图 7-2 内存对齐统计

得到内存访问对齐统计如图 7-2。大部分的内存访问操作均是对齐的，这是因为编译器会自动将所有的操作根据系统的字长对齐，但是仍然存在一小部分的非对其操作，这一份在内存管理系统中主要是因为空闲空间链表是精确到字节的，因此当完成空间分配后，一部分的地址访问可能是不对齐的，例如上面的代码例子中，分配 30 字节显然不对齐，这与内存管理的实现强相关，在我们的系统中并没有不对齐产生的性能开销，因此内存系统并未考虑这一方面。

注意到 LOAD 指令占用最多，分支指令其次。如图 7-2，内存对齐次数为 5 万多次得到操作流水，如图 7-3，注意到 mmu 的启动时间区间和代码的 10 次循环互相印



活时间段被分成了多个小节。



图 7-4 内存对齐次数柱状图

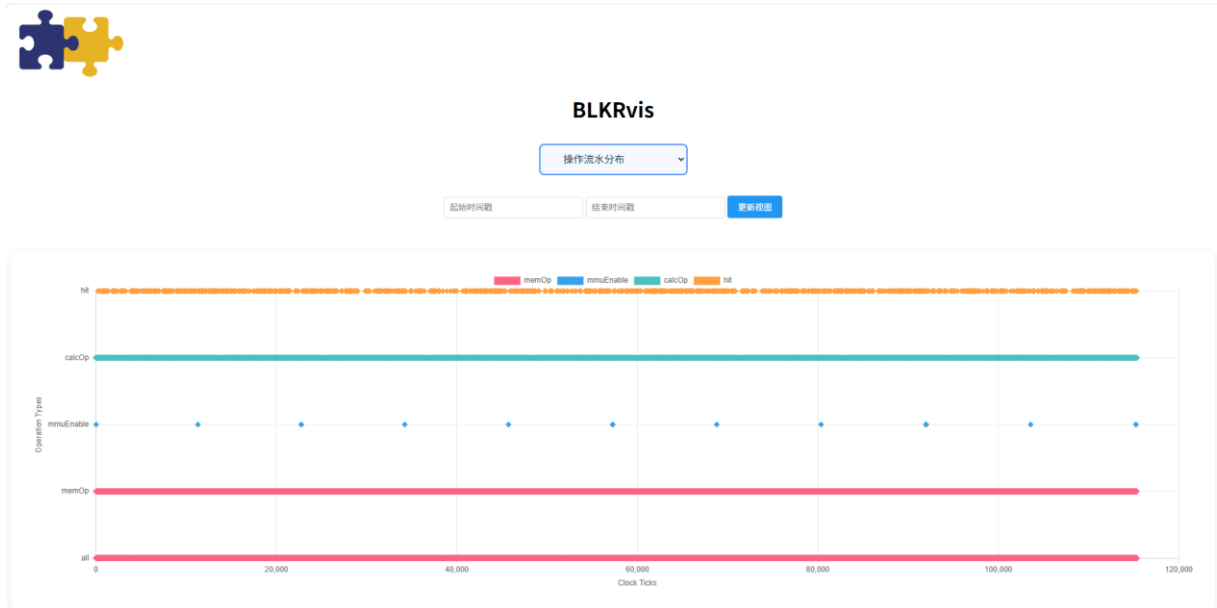


图 7-5 操作流水

参考文献

- [1] KIMOVSKI D, et al. Beyond Von Neumann in the Computing Continuum: Architectures, Applications, and Future Directions[J]. *IEEE Internet Computing*, 2024, 28(3): 6-16.
- [2] QURESHI Y M, et al. Gem5-X: A Many-core Heterogeneous Simulation Platform for Architectural Exploration and Optimization[J]. *ACM Trans. Archit. Code Optim.*, 2021, 18(4): Article 44.
- [3] PARASHAR A, et al. Timeloop: A Systematic Approach to DNN Accelerator Evaluation[C]// *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. Madison, WI, USA, 2019.
- [4] Waterman A, Lee Y, Patterson D, et al. The RISC-V Instruction Set Manual. Volume 1: User-Level ISA, Version 2.0[J]. 2014.
- [5] PATTERSON D, WATERMAN A. *The RISC-V Reader: An Open Architecture Atlas*[M]. 1st ed. Berkeley, CA, USA: Strawberry Canyon LLC, 2017.
- [6] ASANOVIĆ K, et al. *The Rocket Chip Generator*[R]. UCB/EECS-2016-17, 2016.
- [7] HENNESSY J L, PATTERSON D A. A New Golden Age for Computer Architecture[J]. *Communications of the ACM*, 2019, 62(2): 48-60.
- [8] RAMACHANDRAN U, LEAHY W D. An integrated approach to teaching computer systems architecture[C]// *Proceedings of the 2007 workshop on Computer architecture education (WCAE '07)*. New York, NY, USA: Association for Computing Machinery, 2007: 38-43.
- [9] PETERSEN M B. Ripes: A Visual Computer Architecture Simulator[C]// *2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE)*. Raleigh, NC, USA, 2021: 1-8.
- [10] NAVARRO-TORRERO P, MARTÍNEZ-RODRÍGUEZ M C, BARRIGA-BARROS Á, BROX P. Full Open-Source Implementation of an Academic RISC-V on FPGA[C]// *XVI Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (TAE)*. Malaga, Spain, 2024: 1-5.
- [11] HARRIS S L, HARRIS D. Digital Design and RISC-V Computer Architecture Textbook[C]// *2021 ACM/IEEE Workshop on Computer Architecture Education (WCAE)*. Raleigh, NC, USA, 2021: 1-5.
- [12] 向明尚,刘延军,刘志刚,唐克.基于 EDA 的计算机硬件教学仿真平台设计及应用[J].大庆石油学院学报,2011,35(5):92-96121
- [13] 屈泳,阮小军,王三华.计算机硬件虚拟仿真实验平台的建设与设计[J].实验室研究与探索,2018,37(12):116-119
- [14] BRUSCHI N, HAUGOU G, TAGLIAVINI G, et al. GVSoc: A Highly Configurable, Fast and Accurate

Full-Platform Simulator for RISC-V based IoT Processors[J]. 2022.

[15] HAASE J, ALI M, GÖHRINGER D. Unlocking the Potential of RISC-V Heterogeneous MPSoC: A PANACA-Based Approach to Simulation and Modeling[C]// SILVANO C, PILATO C, REICHENBACH M, eds. *Embedded Computer Systems: Architectures, Modeling, and Simulation*. Cham: Springer, 2023: [269–282]. (Lecture Notes in Computer Science, 14385).